

GOVERNO DO ESTADO DO RIO GRANDE DO NORTE

Programa de Capacitação em Python e Django para Servidores Públicos

MÓDULO 1

Fundamentos de Python

Apostila do participante · Etapas 1 a 10

Conteúdo: ambiente de programação, variáveis e tipos, condicionais, repetição, listas, dicionários, funções, strings e exceções, arquivos, módulos e ambientes virtuais.

Instrutor(a): Hideljundes Macêdo Paulino

Elaborado em conjunto com a Claude.ia Opus 5.5

Natal/RN · 2026

Sumário

Sumário.....	1
Como usar esta apostila	2
Etapa 1 — Boas-vindas e ambiente de programação	4
Etapa 2 — Variáveis, tipos de dados e operadores.....	9
Etapa 3 — Estruturas condicionais.....	15
Etapa 4 — Estruturas de repetição.....	19
Etapa 5 — Listas e tuplas.....	24
Etapa 6 — Dicionários e conjuntos.....	29
Etapa 7 — Funções.....	33
Etapa 8 — Strings em profundidade e tratamento de erros.....	38
Etapa 9 — Manipulação de arquivos (TXT, CSV e JSON)	44
Etapa 10 — Módulos, pacotes e ambientes virtuais.....	50
Apêndice A — Gabarito comentado.....	58
Apêndice B — Atalhos essenciais do VS Code.....	66
Apêndice C — Guia de mensagens de erro	67
Apêndice D — Glossário	68
Apêndice E — Para continuar estudando.....	70

Como usar esta apostila

Esta apostila acompanha o **Módulo 1 — Fundamentos de Python** do programa de capacitação em Python e Django para servidores públicos do Governo do Estado do Rio Grande do Norte. Ela foi escrita para quem **nunca programou**, mas também traz dicas que ajudam quem já tem alguma experiência a consolidar boas práticas desde o início.

Cada capítulo corresponde a uma etapa. A leitura do capítulo antes ou depois da Etapa reforça o que foi visto em sala, e os exercícios são a parte mais importante: **programação se aprende programando**.

Estrutura de cada capítulo

- **Objetivos da Etapa:** o que você será capaz de fazer ao final.
- **Conteúdo com exemplos:** explicações curtas seguidas de código que você deve digitar e executar.
- **Caixas de destaque:** dicas, alertas de erros comuns, curiosidades e aplicações no serviço público.
- **Exercícios:** três exercícios de fixação (com gabarito comentado no Apêndice A) e um desafio opcional.
- **Checklist:** uma autoavaliação rápida para saber se você está pronto para a próxima Etapa.

Legenda das caixas de destaque

✓ DICA

Atalhos, boas práticas e truques que economizam tempo.

⚠ ATENÇÃO

Erros muito comuns entre iniciantes e como evitá-los.

◆ VOCÊ SABIA?

Curiosidades e aprofundamentos opcionais.

★ NO SERVIÇO PÚBLICO

Exemplos de como o conteúdo se aplica ao dia a dia da administração pública.

Como ler os blocos de código

Os blocos com fundo cinza contêm **código Python** que você deve digitar no seu editor. Os blocos com fundo escuro mostram a **saída**, isto é, o que aparece na tela quando o programa é executado.

```
# Isto é código Python  
print("Olá!")
```

```
Olá!
```

Quando aparecer o símbolo `>>>` no início da linha, o código está sendo digitado no **modo interativo** do Python (o REPL), que você conhecerá na Etapa 1.

Dez conselhos para aprender mais rápido

1. **Digite o código, não copie e cole.** Digitar treina seus dedos e seu olhar para os detalhes (parênteses, dois-pontos, aspas).
2. **Pratique um pouco todos os dias.** Vinte minutos diários rendem muito mais do que três horas no fim de semana.
3. **Erre sem medo.** Mensagens de erro não são broncas: são pistas. Aprender a lê-las é metade do caminho.
4. **Modifique os exemplos.** Depois que um exemplo funcionar, troque valores, acrescente linhas, tente quebrá-lo de propósito.
5. **Use `print()` para investigar.** Quando não entender o que o programa está fazendo, imprima os valores das variáveis.
6. **Resolva o problema no papel primeiro.** Se você não sabe explicar a solução em português, ainda não é hora de escrever código.
7. **Divida problemas grandes em pequenos.** Faça funcionar uma parte de cada vez.
8. **Leia código dos outros.** Os gabaritos e os exemplos da documentação oficial são ótimas fontes.
9. **Pergunte bem.** Ao pedir ajuda, mostre o código, a mensagem de erro completa e o que você esperava que acontecesse.
10. **Use a inteligência artificial como tutora, não como muleta.** Peça explicações e dicas; evite pedir a solução pronta dos exercícios, porque é justamente o esforço de resolver que gera o aprendizado.

Etapa 1 — Boas-vindas e ambiente de programação

Objetivos da Etapa

- Entender o que é programar e por que o Python é uma boa escolha.
- Instalar o Python e o Visual Studio Code.
- Usar o modo interativo (REPL) como calculadora.
- Criar e executar o seu primeiro programa.
- Ler uma mensagem de erro sem pânico.

1.1 O que é programar?

Programar é escrever **instruções precisas** para que o computador execute uma tarefa. O computador é extremamente rápido, mas não tem bom senso: ele faz exatamente o que foi escrito, nem mais nem menos. Por isso, programar é, antes de tudo, um exercício de **clareza de raciocínio**.

Pense em uma receita de bolo. Ela tem ingredientes (os **dados**), passos em ordem (as **instruções**), decisões ("se a massa estiver mole, acrescente farinha") e repetições ("bata até ficar homogêneo"). Todo programa é construído com essas mesmas peças, e você vai aprender cada uma delas neste módulo.

1.2 Por que Python?

- **Sintaxe simples e legível:** o código Python se parece com inglês estruturado, o que facilita o aprendizado.
- **Versátil:** serve para automação de tarefas, análise de dados, inteligência artificial e sistemas web (com o Django, que estudaremos no Módulo 3).
- **Gratuito e de código aberto:** sem custo de licença, o que é relevante para o orçamento público.
- **Comunidade enorme:** para praticamente qualquer dúvida, alguém já perguntou e respondeu na internet.

★ NO SERVIÇO PÚBLICO

Python é usado por órgãos públicos para automatizar a consolidação de planilhas, gerar relatórios, cruzar bases de dados, consumir dados abertos e desenvolver sistemas internos. Uma tarefa manual que consome horas toda semana pode, muitas vezes, virar um script que roda em segundos.

1.3 Instalando o Python

No Windows

1. Acesse **python.org**, clique em **Downloads** e baixe a versão mais recente do Python 3.

2. Execute o instalador. **Antes de clicar em "Install Now"**, marque a opção **"Add python.exe to PATH"** na parte de baixo da janela.
3. Conclua a instalação.
4. Abra o **Prompt de Comando** (tecle Windows, digite cmd e pressione Enter) e digite o comando abaixo.

```
python --version
```

```
Python 3.13.1
```

⚠ ATENÇÃO

Se você esqueceu de marcar **"Add python.exe to PATH"**, o comando python não será reconhecido. A solução mais simples é executar o instalador novamente, escolher **Modify** (ou desinstalar e reinstalar) e marcar a opção. Esse é o problema de instalação mais comum de todos.

No Linux e no macOS

A maioria das distribuições Linux já vem com Python instalado. Nesses sistemas, o comando costuma ser **python3** em vez de python.

```
python3 --version
```

✓ DICA

O número exato da versão não importa muito neste curso. Qualquer versão a partir do **Python 3.10** funciona com todos os exemplos. Evite apenas o Python 2, que foi descontinuado em 2020.

⚠ ATENÇÃO

Em computadores corporativos, a instalação de programas pode exigir permissão de administrador. Se for o seu caso, procure a equipe de TI do seu órgão **antes** da primeira Etapa.

1.4 Instalando o Visual Studio Code

O **Visual Studio Code** (VS Code) é um editor de código gratuito e muito popular. Ele colore o código, aponta erros e ajuda a completar comandos.

1. Baixe em **code.visualstudio.com** e instale.
2. Abra o VS Code, clique no ícone de **Extensões** na barra lateral (quatro quadradinhos) ou tecle **Ctrl+Shift+X**.
3. Pesquise por **Python** e instale a extensão publicada pela **Microsoft**.

4. Crie uma pasta no seu computador chamada **curso-python** e abra-a no VS Code pelo menu **File > Open Folder**.

✓ DICA

Ative o salvamento automático em **File > Auto Save**. Um dos erros mais frequentes de iniciantes é alterar o código, esquecer de salvar e executar a versão antiga, ficando sem entender por que nada mudou.

1.5 O modo interativo (REPL)

Digite **python** (ou **python3**) no terminal e pressione Enter. Aparecerá o sinal **>>>**, indicando que o Python está esperando um comando. Esse modo se chama **REPL** (Read-Eval-Print Loop): ele lê o que você digita, avalia, mostra o resultado e espera o próximo comando.

```
>>> 2 + 3
5
>>> 10 * 4
40
>>> 7 / 2
3.5
>>> 2 ** 10
1024
```

Para sair do REPL, digite **exit()** e pressione Enter.

✓ DICA

O REPL é o seu laboratório. Sempre que tiver dúvida sobre como algo funciona, teste ali. É muito mais rápido do que criar um arquivo só para isso.

1.6 Seu primeiro programa

No VS Code, crie um arquivo chamado **ola.py** dentro da pasta **curso-python** e digite:

```
# Meu primeiro programa em Python
print("Olá, servidor!")
print("Bem-vindo ao curso de Python.")
```

Para executar, abra o terminal do VS Code (menu **Terminal > New Terminal** ou **Ctrl+`**) e digite:

```
python ola.py
```

```
Olá, servidor!
Bem-vindo ao curso de Python.
```

A função **print()** mostra informações na tela. O texto entre aspas é chamado de **string**. A linha que começa com **#** é um **comentário**: o Python a ignora, mas ela serve para explicar o código a quem for lê-lo (inclusive você, daqui a seis meses).

✓ DICA

Você também pode executar o arquivo clicando no botão de **Play** (triângulo) no canto superior direito do VS Code.

⚠ ATENÇÃO

Evite espaços, acentos e letras maiúsculas nos nomes de arquivos. Prefira **relatorio_mensal.py** a **Relatório Mensal.py**. E nunca dê ao seu arquivo o nome de um módulo do Python, como **random.py** ou **math.py**: isso causa erros estranhos mais tarde.

1.7 Lendo mensagens de erro

Erro faz parte. Veja o que acontece se escrevermos **prin** no lugar de **print**:

```
prin("Olá")
```

```
Traceback (most recent call last):
  File "ola.py", line 1, in <module>
    prin("Olá")
    ^^^^^
NameError: name 'prin' is not defined. Did you mean: 'print'?
```

Essa mensagem se chama **Traceback**. A regra de ouro é **lê-la de baixo para cima**:

- A **última linha** diz o tipo do erro (NameError) e a explicação: o nome prin não está definido. O Python ainda sugere: você quis dizer print?
- As linhas acima mostram **onde** o erro aconteceu: arquivo ola.py, linha 1.

✓ DICA

Copie a última linha da mensagem de erro e pesquise na internet. Em 90% dos casos você encontrará a explicação em minutos. No Apêndice C há uma tabela com os erros mais comuns e suas causas.

◆ VOCÊ SABIA?

Na Etapa 10 você aprenderá a criar **ambientes virtuais**, que isolam as bibliotecas de cada projeto. Por enquanto, usaremos o Python da forma como foi instalado.

Exercícios

Exercício 1.1 — Use o REPL para calcular: (a) quanto é 15% de 3.200; (b) quantas horas há em 30 dias; (c) o resultado de 2 elevado a 20.

Exercício 1.2 — Crie o arquivo **apresentacao.py** que mostre na tela, em linhas separadas, o seu nome, o seu órgão e o seu setor.

Exercício 1.3 — O código abaixo tem dois erros. Execute-o, leia as mensagens e corrija-o.

```
print("Secretaria de Administração")
Print("Governo do RN")
```

Desafio — Usando vários `print()`, desenhe na tela uma "moldura" feita com os caracteres `+`, `-` e `|` contendo o texto GOVERNO DO RN no meio.

Checklist da Etapa

- ☐ Consigo abrir o terminal e verificar a versão do Python.
- ☐ Sei entrar e sair do REPL.
- ☐ Consigo criar um arquivo `.py` no VS Code e executá-lo.
- ☐ Sei o que é um comentário.
- ☐ Sei ler uma mensagem de erro de baixo para cima.

Etapa 2 — Variáveis, tipos de dados e operadores

Objetivos da Etapa

- Criar variáveis e escolher bons nomes para elas.
- Conhecer os tipos básicos: int, float, str e bool.
- Fazer cálculos com operadores aritméticos.
- Receber dados do usuário com input() e convertê-los.
- Formatar a saída com f-strings.

2.1 O que é uma variável?

Uma variável é um **nome que aponta para um valor** guardado na memória. Imagine uma etiqueta colada em uma caixa: a etiqueta é o nome, o conteúdo da caixa é o valor.

```
nome = "Maria"
idade = 42
salario = 4850.75
print(nome)
print(idade)
```

```
Maria
42
```

O sinal = significa **atribuição**, e não igualdade matemática. Leia **idade = 42** como "idade **recebe** 42". Por isso, a linha abaixo, que seria absurda em matemática, é perfeitamente válida em Python:

```
contador = 10
contador = contador + 1 # contador recebe o valor antigo mais 1
print(contador)        # 11
```

2.2 Regras e boas práticas para nomes

Regra ou convenção	Exemplo válido	Exemplo inválido ou ruim
Começar com letra ou _	total, _temp	2total
Sem espaços (use _)	valor_diaria	valor diaria
Sem símbolos especiais	cpf_servidor	cpf-servidor, e-mail
Maiúsculas importam	Nome e nome são diferentes	—
Não usar palavras reservadas	classe_servidor	class, if, for
Nomes descritivos (boa prática)	quantidade_processos	q, x1, aux2

Regra ou convenção	Exemplo válido	Exemplo inválido ou ruim
snake_case (padrão Python)	data_admissao	dataAdmissao

✓ DICA

Um bom nome de variável dispensa comentário. Compare `**x = 1412 * 0.075` com `desconto_inss = salario_minimo * aliquota**`. O segundo se explica sozinho.

◆ VOCÊ SABIA?

O Python permite acentos em nomes de variáveis (salário = 10 funciona), mas a prática recomendada é **evitá-los**, por compatibilidade com outras ferramentas e para manter o padrão das equipes.

2.3 Tipos de dados básicos

Tipo	Nome	Exemplos	Uso típico
int	Inteiro	42, -7, 0	Quantidades, anos, matrículas numéricas
float	Ponto flutuante	3.14, 4850.75	Valores com casas decimais
str	String (texto)	"Natal", 'RN'	Nomes, endereços, descrições
bool	Booleano	True, False	Respostas de sim ou não

Use a função **type()** para descobrir o tipo de um valor:

```
>>> type(42)
<class 'int'>
>>> type(4850.75)
<class 'float'>
>>> type("Natal")
<class 'str'>
>>> type(True)
<class 'bool'>
```

⚠ ATENÇÃO

Em Python, o separador decimal é o **ponto**, não a vírgula. Escreva **4850.75**, e não 4850,75. A vírgula tem outro significado na linguagem e produzirá um resultado inesperado.

✓ DICA

Números de CPF, CEP, telefone e matrícula devem ser guardados como **texto (str)**, e não como número. Você não faz contas com eles, e guardá-los como número apagaria zeros à esquerda: o CEP 05890-000 viraria 5890000.

2.4 Operadores aritméticos

Operador	Operação	Exemplo	Resultado
+	Soma	7 + 2	9
-	Subtração	7 - 2	5
*	Multiplicação	7 * 2	14
/	Divisão	7 / 2	3.5
//	Divisão inteira	7 // 2	3
%	Resto da divisão	7 % 2	1
**	Potência	7 ** 2	49

A precedência segue a matemática: primeiro potência, depois multiplicação e divisão, por fim soma e subtração. Use **parênteses** sempre que houver dúvida; eles também deixam o código mais legível.

```
media = (8 + 7 + 9) / 3
print(media) # 8.0
```

✓ DICA

O operador % (resto) é muito útil: **numero % 2 == 0** verifica se um número é par, e **total_minutos % 60** dá os minutos que sobram depois de converter para horas.

2.5 Entrada de dados com input()

A função **input()** mostra uma mensagem e espera o usuário digitar algo. O valor digitado **sempre chega como texto (str)**, mesmo que seja um número.

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

⚠ ATENÇÃO

Este é, provavelmente, o erro mais comum da Etapa 2. Veja o que acontece ao somar dois valores lidos com **input()**:

```
a = input("Primeiro número: ") # usuário digita 10
b = input("Segundo número: ")  # usuário digita 5
print(a + b)
```

O resultado foi **105** porque somar textos significa **juntá-los** (concatenação). A solução é **converter** o texto para número com **int()** ou **float()**:

```
a = int(input("Primeiro número: "))
b = int(input("Segundo número: "))
print(a + b)  # 15
```

Função	Converte para	Exemplo
int()	Inteiro	int("42") → 42
float()	Decimal	float("3.5") → 3.5
str()	Texto	str(42) → "42"
bool()	Booleano	bool(0) → False

2.6 Formatando a saída com f-strings

As **f-strings** são a forma moderna e mais legível de montar textos com valores de variáveis. Basta colocar a letra **f** antes das aspas e as variáveis entre **chaves**:

```
nome = "João"
dias = 3
print(f"O servidor {nome} viajará por {dias} dias.")
```

```
O servidor João viajará por 3 dias.
```

Dentro das chaves você pode fazer contas e controlar a formatação. **:.2f** significa "número com 2 casas decimais":

```
valor = 1234.5
print(f"Total: R$ {valor:.2f}")      # Total: R$ 1234.50
print(f"Total: R$ {valor:,.2f}")    # Total: R$ 1,234.50
print(f"Dobro: {valor * 2}")        # Dobro: 2469.0
```

✓ DICA

Para exibir valores no **formato brasileiro** (1.234,50), um truque simples é formatar no padrão americano e trocar os separadores:

```
def formatar_real(valor):
    texto = f"{valor:,.2f}"          # '1,234.50'
    return "R$ " + texto.replace(",", "X").replace(".", ",").replace("X", ".")

print(formatar_real(1234.5))  # R$ 1.234,50
```

◆ VOCÊ SABIA?

Não se preocupe com a palavra **def** acima: você aprenderá a criar funções na Etapa 7. Por ora, basta copiar a função e usá-la.

2.7 Exemplo completo: calculadora de diárias

★ NO SERVIÇO PÚBLICO

Os valores e regras abaixo são **fictícios**, criados apenas para fins didáticos. Consulte sempre a legislação vigente do seu órgão.

```
# Calculadora de diárias de viagem (valores fictícios)
print("=== CALCULADORA DE DIÁRIAS ===")

nome = input("Nome do servidor: ")
destino = input("Cidade de destino: ")
dias = int(input("Quantidade de dias: "))
valor_diaria = float(input("Valor da diária (R$): "))

total = dias * valor_diaria

print()
print(f"Servidor: {nome}")
print(f"Destino: {destino}")
print(f"Total: R$ {total:.2f}")
```

```
=== CALCULADORA DE DIÁRIAS ===
Nome do servidor: Ana Souza
Cidade de destino: Mossoró
Quantidade de dias: 3
Valor da diária (R$): 180.50

Servidor: Ana Souza
Destino: Mossoró
Total: R$ 541.50
```

◆ VOCÊ SABIA?

Você pode ter notado que $0.1 + 0.2$ no Python dá **0.30000000000000004**. Isso acontece em quase todas as linguagens, por causa da forma como o computador representa decimais. Para cálculos financeiros oficiais, em que cada centavo importa, existe o módulo **decimal**, que você conhecerá mais adiante no curso.

Exercícios

Exercício 2.1 — Escreva um programa que leia o salário bruto de um servidor e o percentual de um reajuste, e mostre o novo salário com duas casas decimais.

Exercício 2.2 — Escreva um programa que leia uma quantidade de minutos e mostre quantas horas e minutos ela representa. Exemplo: 135 minutos = 2 hora(s) e 15 minuto(s). Dica: use // e %.

Exercício 2.3 — Leia o nome de um município e sua população, e mostre a frase: "O município de X tem Y habitantes." Em seguida, mostre também quanto seria 10% dessa população.

Desafio — Leia um valor em reais e mostre a menor quantidade de cédulas de 100, 50, 20, 10, 5 e 2 necessárias para compô-lo (considere apenas valores inteiros).

Checklist da Etapa

- ☐ Sei criar variáveis com nomes descritivos em snake_case.
- ☐ Conheço os tipos int, float, str e bool.
- ☐ Sei por que CPF e CEP devem ser guardados como texto.
- ☐ Sei que input() sempre retorna texto e como convertê-lo.
- ☐ Consigo formatar valores com f-strings e duas casas decimais.

Etapa 3 — Estruturas condicionais

Objetivos da Etapa

- Comparar valores com operadores relacionais.
- Tomar decisões com if, elif e else.
- Entender a importância da indentação em Python.
- Combinar condições com and, or e not.

3.1 Operadores de comparação

Toda decisão em um programa começa com uma **pergunta** cuja resposta é verdadeiro (**True**) ou falso (**False**).

Operador	Significado	Exemplo	Resultado
==	Igual a	5 == 5	True
!=	Diferente de	5 != 3	True
>	Maior que	5 > 8	False
<	Menor que	5 < 8	True
>=	Maior ou igual	5 >= 5	True
<=	Menor ou igual	4 <= 3	False

⚠ ATENÇÃO

Não confunda = (atribuição: guarda um valor) com == (comparação: faz uma pergunta). Escrever **if idade = 18:** gera um `SyntaxError`.

3.2 A estrutura if

```
idade = int(input("Idade: "))

if idade >= 18:
    print("Maior de idade.")
    print("Pode prosseguir com o cadastro.")

print("Fim do programa.")
```

Repare em dois detalhes fundamentais: os **dois-pontos (:)** no fim da linha do if e o **recuo** (indentação) das linhas que pertencem a ele. A última linha, sem recuo, é executada sempre, qualquer que seja a idade.

⚠ ATENÇÃO

Em Python, a **indentação não é enfeite: ela faz parte da sintaxe**. É ela que diz quais linhas pertencem ao bloco do if. Use sempre **4 espaços** (o VS Code faz isso automaticamente quando você tecla Tab). Misturar tabulações e espaços gera o temido **IndentationError**.

3.3 if, else e elif

Use **else** para o caminho alternativo e **elif** (abreviação de "else if") para testar várias condições em sequência. O Python executa **apenas o primeiro bloco** cuja condição for verdadeira.

```
nota = float(input("Nota da avaliação (0 a 10): "))

if nota >= 9:
    conceito = "Excelente"
elif nota >= 7:
    conceito = "Bom"
elif nota >= 5:
    conceito = "Regular"
else:
    conceito = "Insuficiente"

print(f"Conceito: {conceito}")
```

✓ DICA

A ordem dos elif importa. Se o teste **nota >= 5** viesse primeiro, uma nota 9,5 cairia nele e receberia "Regular". Ordene as condições da **mais restritiva para a menos restritiva**.

3.4 Operadores lógicos

Operador	Resultado é True quando...	Exemplo
and	as duas condições são verdadeiras	idade >= 18 and idade < 65
or	pelo menos uma é verdadeira	uf == "RN" or uf == "PB"
not	inverte o valor	not ativo

```
idade = 60
tempo_servico = 32

if idade >= 60 and tempo_servico >= 30:
    print("Atende aos requisitos (regra fictícia).")
else:
    print("Ainda não atende aos requisitos.")
```

✓ DICA

Python permite comparações encadeadas, como na matemática: **18 <= idade < 65** é o mesmo que **idade >= 18 and idade < 65**, só que mais legível.

✓ DICA

Para testar se um valor está entre várias opções, use **in**: **if uf in ("RN", "PB", "CE"):** é muito mais limpo do que três comparações com **or**.

3.5 Valores "verdadeiros" e "falsos"

Em uma condição, o Python considera **falsos** os valores: **False**, **0**, **0.0**, o texto vazio **""**, e estruturas vazias (que veremos adiante). Todo o resto é considerado verdadeiro. Isso permite escrever verificações curtas:

```
nome = input("Nome: ")
if nome:
    print(f"Olá, {nome}!")
else:
    print("Você não digitou nada.")
```

3.6 Expressão condicional (if em uma linha)

```
saldo = -50
situacao = "Regular" if saldo >= 0 else "Pendente"
print(situacao)  # Pendente
```

✓ DICA

Use a forma de uma linha apenas para decisões **simples**, com duas alternativas. Se a lógica for mais complexa, o **if** tradicional é mais claro.

◆ VOCÊ SABIA?

Desde o Python 3.10 existe também o comando **match/case**, útil para comparar um valor com muitas opções fixas (como códigos de status). O **if/elif** continua resolvendo todos os casos deste curso.

3.7 Exemplo completo: classificação de prioridade de processo

★ NO SERVIÇO PÚBLICO

Sistemas de protocolo costumam classificar processos por prioridade. O exemplo abaixo usa regras fictícias para ilustrar decisões combinadas.

```
print("=== CLASSIFICAÇÃO DE PRIORIDADE ===")
dias_parado = int(input("Dias sem movimentação: "))
tem_prazo_judicial = input("Tem prazo judicial? (s/n): ").strip().lower() == "s"
idoso = input("Interessado com 60 anos ou mais? (s/n): ").strip().lower() == "s"

if tem_prazo_judicial:
    prioridade = "URGENTE"
elif idoso or dias_parado > 30:
    prioridade = "ALTA"
elif dias_parado > 15:
    prioridade = "MÉDIA"
else:
    prioridade = "NORMAL"

print(f"Prioridade: {prioridade}")
```

✓ DICA

O trecho `.strip().lower()` remove espaços extras e converte a resposta para minúsculas. Assim, o programa aceita "S", "s" ou "s". Tratar a entrada do usuário com essa tolerância é uma excelente prática; veremos mais na Etapa 8.

Exercícios

Exercício 3.1 — Leia um número inteiro e informe se ele é par ou ímpar.

Exercício 3.2 — Leia a quantidade de horas trabalhadas no mês. Se passar de 160, mostre quantas horas extras foram feitas; caso contrário, mostre "Sem horas extras".

Exercício 3.3 — Leia a sigla de um estado. Se for RN, PB, PE, CE, AL, SE, BA, PI ou MA, mostre "Região Nordeste"; caso contrário, mostre "Outra região". Aceite a sigla em maiúsculas ou minúsculas.

Desafio — Leia três valores e informe se eles podem formar um triângulo (cada lado deve ser menor que a soma dos outros dois) e, em caso positivo, se ele é equilátero, isósceles ou escaleno.

Checklist da Etapa

- ☐ Sei a diferença entre `=` e `==`.
- ☐ Uso dois-pontos e indentação de 4 espaços nos blocos.
- ☐ Sei encadear condições com `elif` na ordem correta.
- ☐ Sei combinar condições com `and`, `or` e `not`.
- ☐ Sei usar `in` para comparar com várias opções.

Etapa 4 — Estruturas de repetição

Objetivos da Etapa

- Repetir instruções com `for` e `range()`.
- Repetir enquanto uma condição for verdadeira com `while`.
- Controlar laços com `break` e `continue`.
- Usar contadores e acumuladores.
- Validar a entrada do usuário com repetição.

4.1 Por que repetir?

Imagine mostrar os números de 1 a 1.000. Com o que sabemos até agora, seriam mil linhas de `print()`. Os **laços de repetição** (ou **loops**) resolvem isso em duas linhas. Eles são a razão pela qual a automação economiza tanto tempo: o computador não se cansa de repetir.

4.2 O laço `for` com `range()`

```
for numero in range(1, 6):  
    print(numero)
```

```
1  
2  
3  
4  
5
```

Leia como: "para cada número no intervalo de 1 a 6, mostre o número". A variável **numero** assume um valor diferente a cada volta (chamada de **iteração**).

Forma	Gera	Observação
<code>range(5)</code>	0, 1, 2, 3, 4	Começa em 0 por padrão
<code>range(1, 6)</code>	1, 2, 3, 4, 5	O fim não é incluído
<code>range(0, 20, 5)</code>	0, 5, 10, 15	O terceiro valor é o passo
<code>range(10, 0, -1)</code>	10, 9, ..., 1	Passo negativo faz contagem regressiva

⚠ ATENÇÃO

O valor final do `range()` **nunca é incluído**. `range(1, 10)` vai de 1 a 9. Para incluir o 10, escreva `range(1, 11)`. Esse "erro por um" é clássico em programação.

O `for` também percorre os caracteres de um texto, e, como você verá na Etapa 5, os itens de uma lista:

```
for letra in "RN":
    print(letra)
```

4.3 Contadores e acumuladores

Dois padrões aparecem em quase todo programa com laços: o **contador** (quantas vezes algo aconteceu) e o **acumulador** (a soma de valores).

```
total = 0          # acumulador começa em zero
quantidade = 0     # contador começa em zero

for mes in range(1, 13):
    gasto = float(input(f"Gasto do mês {mes}: R$ "))
    total = total + gasto
    if gasto > 1000:
        quantidade = quantidade + 1

print(f"Total no ano: R$ {total:.2f}")
print(f"Meses acima de R$ 1.000: {quantidade}")
```

✓ DICA

Existe uma forma abreviada para atualizar variáveis: **total += gasto** é o mesmo que `total = total + gasto`. Também existem `-=`, `*=` e `/=`.

⚠ ATENÇÃO

Inicialize acumuladores e contadores **antes** do laço. Se colocar **total = 0** dentro do `for`, ele será zerado a cada volta e o resultado final será apenas o último valor.

4.4 O laço while

O **while** repete enquanto uma condição for verdadeira. Use-o quando você **não sabe de antemão** quantas repetições serão necessárias.

```
saldo = 1000
meses = 0
while saldo > 0:
    saldo -= 150
    meses += 1
print(f"O saldo acaba em {meses} meses.")
```

⚠ ATENÇÃO

Se a condição do `while` nunca se tornar falsa, o programa fica preso em um **laço infinito**. Para interrompê-lo, tecla **Ctrl+C** no terminal. Sempre verifique se algo dentro do laço altera a condição.

4.5 `break` e `continue`

- **`break`**: encerra o laço imediatamente.
- **`continue`**: pula o restante da volta atual e vai para a próxima.

A combinação **`while True`** com **`break`** é o padrão mais usado para **validar entradas**: repita a pergunta até a resposta ser válida.

```
while True:
    dias = int(input("Quantidade de dias (1 a 30): "))
    if 1 <= dias <= 30:
        break
    print("Valor inválido. Tente novamente.")

print(f"Você informou {dias} dias.")
```

```
# Mostra apenas os meses que não são de férias
for mes in range(1, 13):
    if mes == 7:
        continue
    print(f"Mês {mes}: expediente normal")
```

4.6 Laços aninhados

Um laço pode ficar dentro de outro. Para cada volta do laço de fora, o de dentro executa todas as suas voltas:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("---")
```

4.7 Exemplo completo: tabela de progressão salarial

★ NO SERVIÇO PÚBLICO

Projeções de folha de pagamento são um uso comum de laços em planejamento orçamentário. Os percentuais abaixo são fictícios.

```
salario = float(input("Salário atual: R$ "))
reajuste = float(input("Reajuste anual (%): "))
anos = int(input("Projetar por quantos anos? "))

print(f"{'Ano':>4} | {'Salário':>12}")
print("-" * 20)
for ano in range(1, anos + 1):
    salario = salario * (1 + reajuste / 100)
    print(f"{'ano':>4} | R$ {salario:>9.2f}")
```

```
Salário atual: R$ 3000
Reajuste anual (%): 5
Projetar por quantos anos? 3
Ano |      Salário
-----
 1 | R$   3150.00
 2 | R$   3307.50
 3 | R$   3472.88
```

✓ DICA

Nas f-strings, `>9` alinha o valor à direita em 9 posições, e `<9` alinha à esquerda. Isso permite montar tabelas bem organizadas no terminal. Já `**"- " * 20**` repete o caractere 20 vezes.

4.8 Depurando laços com print()

Quando um laço não se comporta como esperado, coloque um `print()` dentro dele para ver o que acontece a cada volta. Essa técnica simples resolve a maioria dos problemas:

```
total = 0
for n in range(1, 5):
    total += n
    print(f"[DEBUG] n={n}, total={total}") # remova depois de entender
```

✓ DICA

O VS Code tem um **depurador** completo: clique à esquerda do número de uma linha para criar um ponto de parada (bolinha vermelha) e tecla **F5**. O programa pausa ali e você pode ver o valor de cada variável. Vale a pena experimentar.

Exercícios

Exercício 4.1 — Mostre a tabuada de um número lido do usuário, de 1 a 10.

Exercício 4.2 — Leia 5 valores de despesas e mostre o total, a média e a maior despesa.

Exercício 4.3 — Peça uma senha repetidamente até o usuário digitar "gov2026". Ao final, mostre quantas tentativas foram necessárias.

Desafio — Um órgão tem 1.200 processos em estoque. A cada mês entram 150 novos e são concluídos 230. Em quantos meses o estoque será zerado? Mostre a evolução mês a mês.

Checklist da Etapa

- ☐ Sei usar `range()` com início, fim e passo.
- ☐ Sei quando usar `for` e quando usar `while`.
- ☐ Inicializo acumuladores e contadores antes do laço.
- ☐ Sei validar entradas com `while True` e `break`.
- ☐ Sei interromper um laço infinito com `Ctrl+C`.

Etapa 5 — Listas e tuplas

Objetivos da Etapa

- Guardar vários valores em uma lista.
- Acessar itens por índice e fatiar listas.
- Adicionar, remover e ordenar itens.
- Percorrer listas com for e enumerate().
- Criar listas com list comprehension.
- Entender quando usar tuplas.

5.1 Criando listas

Uma **lista** guarda vários valores em uma única variável, em ordem. Ela é escrita entre **colchetes**, com os itens separados por vírgula.

```
municipios = ["Natal", "Mossoró", "Parnamirim", "Caicó"]
notas = [8.5, 7.0, 9.2]
vazia = []

print(municipios)
print(len(municipios))  # quantidade de itens: 4
```

5.2 Índices e fatiamento

Cada item tem uma posição, chamada **índice**, que **começa em 0**. Índices negativos contam a partir do fim.

Item	"Natal"	"Mossoró"	"Parnamirim"	"Caicó"
Índice positivo	0	1	2	3
Índice negativo	-4	-3	-2	-1

```
print(municipios[0])  # Natal
print(municipios[-1]) # Caicó (o último)
print(municipios[1:3]) # ['Mossoró', 'Parnamirim']
print(municipios[:2])  # ['Natal', 'Mossoró']
print(municipios[2:])  # ['Parnamirim', 'Caicó']
```

⚠ ATENÇÃO

Acessar um índice que não existe, como `municipios[10]`, gera **IndexError: list index out of range**. Lembre-se de que uma lista com 4 itens vai do índice 0 ao 3.

✓ DICA

No fatiamento [**início:fim**], a mesma regra do `range()` vale: o item da posição fim **não é incluído**.

5.3 Modificando listas

Método	O que faz	Exemplo
<code>append(x)</code>	Adiciona x no fim	<code>lista.append("Macau")</code>
<code>insert(i, x)</code>	Insere x na posição i	<code>lista.insert(0, "Assu")</code>
<code>remove(x)</code>	Remove a primeira ocorrência de x	<code>lista.remove("Natal")</code>
<code>pop(i)</code>	Remove e devolve o item da posição i (o último, se vazio)	<code>ultimo = lista.pop()</code>
<code>sort()</code>	Ordena a própria lista	<code>lista.sort()</code>
<code>reverse()</code>	Inverte a ordem	<code>lista.reverse()</code>
<code>count(x)</code>	Conta quantas vezes x aparece	<code>lista.count("Natal")</code>
<code>index(x)</code>	Posição da primeira ocorrência	<code>lista.index("Caicó")</code>

```
pendentes = ["2024/001", "2024/007"]
pendentes.append("2024/015")
pendentes[0] = "2024/002"      # troca o primeiro item
print(pendentes)
print("2024/007" in pendentes) # True: verifica se existe
```

5.4 Funções úteis com listas

```
valores = [350.0, 1200.5, 89.9, 640.0]
print(sum(valores))      # soma: 2280.4
print(min(valores))      # menor: 89.9
print(max(valores))      # maior: 1200.5
print(sorted(valores))    # nova lista ordenada, a original não muda
print(sorted(valores, reverse=True)) # ordem decrescente
```

✓ DICA

Diferença importante: **`lista.sort()`** altera a própria lista e não devolve nada; **`sorted(lista)`** devolve uma **nova** lista ordenada e preserva a original. Um erro comum é escrever **`lista = lista.sort()`**, que transforma a lista em `None`.

5.5 Percorrendo listas

```
for municipio in municipios:
    print(f"Enviando ofício para {municipio}")
```

Quando precisar da posição e do valor, use **enumerate()**:

```
for posicao, municipio in enumerate(municipios, start=1):
    print(f"{posicao}. {municipio}")
```

```
1. Natal
2. Mossoró
3. Parnamirim
4. Caicó
```

✓ DICA

Evite o estilo **for i in range(len(lista))** para percorrer listas. É válido, mas **for item in lista** ou **enumerate()** são mais legíveis e considerados mais "pythônicos".

5.6 List comprehension

É uma forma compacta de **criar uma lista a partir de outra**, transformando ou filtrando itens. Compare as duas versões:

```
salarios = [2500, 4800, 7200, 3100]

# Forma tradicional
reajustados = []
for s in salarios:
    reajustados.append(s * 1.05)

# Com list comprehension
reajustados = [s * 1.05 for s in salarios]

# Com filtro: apenas salários acima de 3.000
acima = [s for s in salarios if s > 3000]
print(acima)  # [4800, 7200, 3100]
```

✓ DICA

Leia a list comprehension como uma frase: **"s vezes 1,05, para cada s em salários"**. Se a expressão ficar longa ou difícil de ler, prefira o for tradicional. Clareza vale mais do que esperteza.

5.7 Cuidado ao copiar listas

```
original = [1, 2, 3]
copia = original      # NÃO copia! Os dois nomes apontam para a mesma lista
copia.append(4)
print(original)        # [1, 2, 3, 4] (a original também mudou!)

copia_real = original.copy()  # esta sim é uma cópia independente
```

⚠ ATENÇÃO

A atribuição **b = a** não cria uma nova lista: apenas cola uma segunda etiqueta na mesma caixa. Para copiar, use **a.copy()** ou **list(a)**.

5.8 Tuplas

Uma **tupla** é como uma lista, mas **imutável**: depois de criada, não pode ser alterada. É escrita entre **parênteses**.

```
coordenadas_natal = (-5.79, -35.21)
meses_trimestre = ("jan", "fev", "mar")

latitude, longitude = coordenadas_natal  # desempacotamento
print(latitude)    # -5.79
```

✓ DICA

Use tuplas para dados que **não devem mudar** e que formam um conjunto natural, como coordenadas, datas (dia, mês, ano) ou opções fixas. Use listas para coleções que crescem ou mudam.

5.9 Exemplo completo: controle de processos pendentes

```
processos = []

while True:
    numero = input("Número do processo (Enter para encerrar): ").strip()
    if not numero:
        break
    if numero in processos:
        print("Processo já cadastrado!")
    else:
        processos.append(numero)

processos.sort()
print(f"\n{len(processos)} processo(s) pendente(s):")
for i, p in enumerate(processos, start=1):
    print(f"  {i}. {p}")
```

◆ VOCÊ SABIA?

O `\n` dentro de um texto representa uma **quebra de linha**. Ele é útil para dar espaçamento na saída.

Exercícios

Exercício 5.1 — Crie uma lista com as notas de 6 servidores em uma avaliação. Mostre a maior nota, a menor, a média e quantas notas ficaram acima da média.

Exercício 5.2 — Leia nomes de municípios até o usuário digitar "fim". Mostre a lista em ordem alfabética, sem repetir municípios.

Exercício 5.3 — A partir da lista `valores = [120, 45, 980, 300, 15, 760]`, use list comprehension para criar: (a) uma lista só com os valores acima de 100; (b) uma lista com todos os valores acrescidos de 10%.

Desafio — Leia os valores de 12 meses de consumo de energia de um prédio público. Mostre o mês de maior consumo (pelo nome: janeiro, fevereiro...), usando uma segunda lista com os nomes dos meses.

Checklist da Etapa

- ☐ Sei que os índices começam em 0 e sei usar índices negativos.
- ☐ Sei adicionar, remover e ordenar itens.
- ☐ Conheço a diferença entre `sort()` e `sorted()`.
- ☐ Sei usar `enumerate()` e list comprehension.
- ☐ Sei copiar uma lista corretamente e quando usar tuplas.

Etapa 6 — Dicionários e conjuntos

Objetivos da Etapa

- Guardar dados no formato chave e valor com dicionários.
- Acessar, incluir, alterar e remover informações com segurança.
- Representar registros com listas de dicionários.
- Eliminar duplicatas e comparar grupos com conjuntos.

6.1 O que é um dicionário?

Em uma lista, você acessa os itens pela posição. Em um **dicionário**, você acessa pelo **nome** (a **chave**). É como uma ficha cadastral: cada campo tem um rótulo e um conteúdo.

```
servidor = {  
    "matricula": "123456",  
    "nome": "Carla Medeiros",  
    "cargo": "Analista",  
    "salario": 5200.00,  
    "ativo": True,  
}  
  
print(servidor["nome"])    # Carla Medeiros  
print(servidor["salario"]) # 5200.0
```

✓ DICA

Repare na vírgula depois do último item. Ela é opcional, mas é uma boa prática em estruturas escritas em várias linhas: facilita acrescentar novos itens e deixa as comparações de versões (no Git) mais limpas.

6.2 Operações básicas

```
servidor["lotacao"] = "SEAD"    # adiciona uma nova chave  
servidor["cargo"] = "Coordenador" # altera um valor existente  
del servidor["ativo"]          # remove uma chave  
  
print("cpf" in servidor)        # False: verifica se a chave existe  
print(len(servidor))            # quantidade de chaves
```

⚠ ATENÇÃO

Acessar uma chave que não existe, como `servidor["cpf"]`, gera **KeyError**. Para evitar isso, use **get()**, que devolve `None` (ou um valor padrão que você escolher) quando a chave não existe:

```
print(servidor.get("cpf"))           # None
print(servidor.get("cpf", "não informado")) # não informado
```

6.3 Percorrendo dicionários

```
for chave in servidor:               # percorre as chaves
    print(chave)

for chave, valor in servidor.items(): # percorre chaves e valores
    print(f"{chave}: {valor}")

print(list(servidor.keys()))         # só as chaves
print(list(servidor.values()))       # só os valores
```

✓ DICA

items() é a forma mais usada de percorrer dicionários. Grave o padrão **for chave, valor in dicionario.items():**.

6.4 Listas de dicionários: a estrutura de uma tabela

Uma **lista de dicionários** é a forma natural de representar uma tabela: cada dicionário é uma **linha** e cada chave é uma **coluna**. Essa estrutura aparece o tempo todo: em arquivos CSV, em APIs e, mais tarde, nos resultados de consultas do Django.

```
servidores = [
    {"nome": "Carla", "setor": "RH", "salario": 5200},
    {"nome": "Paulo", "setor": "TI", "salario": 6100},
    {"nome": "Rita", "setor": "RH", "salario": 4700},
]

for s in servidores:
    print(f"{s['nome']:<8} {s['setor']:<4} R$ {s['salario']:.2f}")

folha_rh = sum(s["salario"] for s in servidores if s["setor"] == "RH")
print(f"Folha do RH: R$ {folha_rh:.2f}")
```

⚠ ATENÇÃO

Dentro de uma f-string delimitada por aspas duplas, use **aspas simples** para as chaves do dicionário: **f"{s['nome']}"**. Usar o mesmo tipo de aspas por dentro e por fora causa erro em versões do Python anteriores à 3.12.

6.5 Contando ocorrências

Um uso clássico de dicionários é contar quantas vezes cada valor aparece:

```
setores = ["RH", "TI", "RH", "Jurídico", "TI", "RH"]

contagem = {}
for setor in setores:
    contagem[setor] = contagem.get(setor, 0) + 1

print(contagem)  # {'RH': 3, 'TI': 2, 'Jurídico': 1}
```

◆ VOCÊ SABIA?

A biblioteca padrão do Python tem um atalho pronto para isso: **from collections import Counter** e depois **Counter(setores)**. Você aprenderá a usar o import na Etapa 10.

6.6 Dicionários aninhados

```
orgaos = {
    "SEAD": {"nome": "Secretaria de Administração", "servidores": 850},
    "SESAP": {"nome": "Secretaria de Saúde Pública", "servidores": 12000},
}
print(orgaos["SEAD"]["nome"])
```

6.7 Conjuntos (set)

Um **conjunto** é uma coleção **sem itens repetidos** e **sem ordem definida**. É escrito entre chaves, mas sem o formato chave: valor.

```
cpfs = ["111", "222", "111", "333", "222"]
unicos = set(cpfs)
print(unicos)      # {'111', '222', '333'} (a ordem pode variar)
print(len(unicos)) # 3
```

Conjuntos permitem as operações da matemática, o que é muito útil para **cruzar listas**:

```
turma_python = {"Ana", "Bruno", "Carla", "Davi"}
turma_excel = {"Carla", "Davi", "Elisa"}

print(turma_python & turma_excel)  # interseção: {'Carla', 'Davi'}
print(turma_python | turma_excel)  # união: todos, sem repetir
print(turma_python - turma_excel)  # diferença: {'Ana', 'Bruno'}
```

★ NO SERVIÇO PÚBLICO

Cruzamentos como "quem está na folha de pagamento mas não está no cadastro de frequência" ou "quais CPFs aparecem nas duas bases" são resolvidos em uma linha com conjuntos, mesmo com dezenas de milhares de registros.

⚠️ ATENÇÃO

Para criar um conjunto vazio, use `set()`. Escrever `{}` cria um **dicionário** vazio, e não um conjunto.

6.8 Qual estrutura usar?

Estrutura	Sintaxe	Ordenada?	Mutável?	Aceita repetição?	Use para
Lista	[1, 2]	Sim	Sim	Sim	Coleções em sequência
Tupla	(1, 2)	Sim	Não	Sim	Dados fixos
Dicionário	{"a": 1}	Sim*	Sim	Chaves não	Dados com rótulos
Conjunto	{1, 2}	Não	Sim	Não	Itens únicos e cruzamentos

* Desde o Python 3.7, os dicionários mantêm a ordem em que as chaves foram inseridas.

Exercícios

Exercício 6.1 — Crie um dicionário com os dados de um veículo da frota (placa, modelo, ano, quilometragem). Mostre todos os dados com `items()` e depois atualize a quilometragem com um valor lido do usuário.

Exercício 6.2 — Dada a lista `["Natal", "Mossoró", "Natal", "Caicó", "Natal", "Mossoró"]` de cidades de destino de viagens, conte quantas viagens foram feitas para cada cidade.

Exercício 6.3 — Crie um pequeno cadastro: leia matrícula e nome de servidores até a matrícula ficar vazia, guardando tudo em um dicionário `{matrícula: nome}`. Depois, peça uma matrícula e mostre o nome correspondente, ou "Matrícula não encontrada".

Desafio — Dadas duas listas de CPFs (uma da folha de pagamento e outra do registro de frequência), mostre: os CPFs presentes nas duas, os que estão só na folha e os que estão só na frequência.

Checklist da Etapa

- ☐ Sei criar dicionários e acessar valores pela chave.
- ☐ Uso `get()` para evitar `KeyError`.
- ☐ Sei percorrer dicionários com `items()`.
- ☐ Entendo listas de dicionários como tabelas.
- ☐ Sei usar conjuntos para remover duplicatas e cruzar dados.

Etapa 7 — Funções

Objetivos da Etapa

- Criar funções com `def`.
- Usar parâmetros, valores padrão e argumentos nomeados.
- Entender a diferença entre `return` e `print()`.
- Compreender o escopo de variáveis.
- Documentar funções com docstrings e anotações de tipo.

7.1 Por que criar funções?

Você já usou várias funções prontas: `print()`, `input()`, `len()`, `sum()`. Agora vai criar as suas. Uma **função** é um bloco de código com nome, que pode ser **reutilizado** quantas vezes for necessário.

- **Evita repetição:** se você copia e cola o mesmo trecho em vários lugares, ele deveria ser uma função (princípio DRY: *Don't Repeat Yourself*).
- **Organiza:** um programa grande vira um conjunto de peças pequenas e compreensíveis.
- **Facilita a manutenção:** para corrigir um cálculo, você altera um único lugar.
- **Facilita os testes:** cada função pode ser testada isoladamente.

7.2 Criando uma função

```
def saudar():  
    print("Bom dia, equipe!")  
  
saudar()    # chamando a função  
saudar()
```

A palavra **def** define a função. Assim como no `if`, há **dois-pontos** e **indentação**. Definir a função não a executa: ela só roda quando é **chamada** pelo nome, com parênteses.

7.3 Parâmetros

```
def saudar(nome, periodo):  
    print(f"{periodo}, {nome}!")  
  
saudar("Ana", "Bom dia")  
saudar("Paulo", "Boa tarde")
```

Parâmetros são as variáveis declaradas na definição (`nome`, `periodo`). **Argumentos** são os valores enviados na chamada (`"Ana"`, `"Bom dia"`).

Valores padrão e argumentos nomeados

```
def saudar(nome, periodo="Olá"):
    print(f"{periodo}, {nome}!")

saudar("Ana")                # Olá, Ana!
saudar("Ana", "Boa noite")   # Boa noite, Ana!
saudar(periodo="Bom dia", nome="Rui") # nomeados: a ordem não importa
```

✓ DICA

Argumentos nomeados deixam as chamadas autoexplicativas. `calcular_diarias(3, 180, True)` é obscuro; `calcular_diarias(dias=3, valor=180, com_pernoite=True)` se explica sozinho.

7.4 return: devolvendo resultados

Até aqui as funções apenas mostravam algo na tela. Na maioria das vezes, porém, queremos que a função **calcule e devolva** um valor, para que ele possa ser usado em outras partes do programa. Para isso serve o **return**.

```
def calcular_desconto(salario, percentual):
    return salario * percentual / 100

desconto = calcular_desconto(5000, 11)
liquido = 5000 - desconto
print(f"Líquido: R$ {liquido:.2f}") # Líquido: R$ 4450.00
```

⚠ ATENÇÃO

print() não é return! Uma função que só faz `print()` mostra o valor na tela, mas **devolve None**. Se você tentar usar o resultado em uma conta, receberá um `TypeError`. Regra prática: funções de **cálculo** usam `return`; deixe o `print()` para a parte do programa que conversa com o usuário.

```
def dobro_errado(x):
    print(x * 2)    # só mostra

def dobro_certo(x):
    return x * 2    # devolve

resultado = dobro_errado(5) # mostra 10, mas resultado vale None
resultado = dobro_certo(5)  # resultado vale 10
```

O `return` também **encerra a função** imediatamente. Isso permite retornar mais cedo quando uma condição é atendida:

```
def classificar_idade(idade):
    if idade < 18:
        return "Menor"
    if idade < 60:
        return "Adulto"
    return "Idoso"
```

Uma função pode devolver **vários valores** de uma vez, na forma de uma tupla:

```
def estatisticas(valores):
    return min(valores), max(valores), sum(valores) / len(valores)

menor, maior, media = estatisticas([10, 20, 30])
```

7.5 Escopo de variáveis

Variáveis criadas **dentro** de uma função são **locais**: existem apenas enquanto a função executa e não são vistas do lado de fora.

```
def calcular():
    resultado = 42  # variável local

calcular()
print(resultado)  # NameError: name 'resultado' is not defined
```

✓ DICA

Se você precisa de um valor calculado dentro da função, **devolva-o com return**. Evite usar a palavra-chave **global** para alterar variáveis externas: isso torna o código difícil de entender e de testar.

7.6 Documentando funções

Uma **docstring** é um texto entre aspas triplas, logo abaixo do def, que explica o que a função faz. As **anotações de tipo** (*type hints*) indicam os tipos esperados dos parâmetros e do retorno.

```
def calcular_diarias(dias: int, valor_diaria: float,
                    meia_diaria: bool = False) -> float:
    """Calcula o valor total de diárias de uma viagem.

    Se meia_diaria for True, acrescenta meia diária pelo dia de retorno.
    """
    total = dias * valor_diaria
    if meia_diaria:
        total += valor_diaria / 2
    return total
```

✓ DICA

Passe o mouse sobre o nome de uma função documentada no VS Code: a docstring aparece como ajuda. E digite **help(calcular_diarias)** no REPL para ver a documentação. As anotações de tipo não mudam o funcionamento do programa, mas ajudam o editor a encontrar erros antes da execução.

7.7 Boas práticas

- **Uma função, uma responsabilidade.** Se o nome precisa de "e" (calcular_e_salvar_e_enviar), divida em várias.
- **Nomes com verbos:** calcular_total, validar_cpf, gerar_relatorio.
- **Funções curtas:** se passar de umas 20 ou 30 linhas, provavelmente dá para dividir.
- **Separe cálculo de interface:** funções que calculam não devem usar input() nem print().

7.8 Exemplo completo: calculadora de diárias refatorada

```
def ler_inteiro(mensagem: str) -> int:
    """Pede um número inteiro até o usuário digitar um valor válido."""
    while True:
        texto = input(mensagem)
        if texto.isdigit():
            return int(texto)
        print("Digite apenas números inteiros.")

def calcular_diarias(dias: int, valor_diaria: float) -> float:
    """Calcula o total de diárias."""
    return dias * valor_diaria

def main():
    nome = input("Servidor: ")
    dias = ler_inteiro("Dias de viagem: ")
    total = calcular_diarias(dias, 180.50)
    print(f"{nome} receberá R$ {total:.2f}")

main()
```

✓ DICA

Organizar o programa com uma função **main()** que coordena as demais é um padrão muito comum. Na Etapa 10, você verá como deixar esse padrão ainda mais profissional.

Exercícios

Exercício 7.1 — Crie a função **eh_par(numero)** que retorne True se o número for par e False caso contrário. Teste com vários valores.

Exercício 7.2 — Crie a função **calcular_imc(peso, altura)** que retorne o IMC, e a função **classificar_imc(imc)** que retorne a classificação (abaixo de 18,5: "Abaixo do peso"; até 24,9: "Normal"; até 29,9: "Sobrepeso"; acima: "Obesidade").

Exercício 7.3 — Crie a função **media_lista(valores)** que receba uma lista de números e retorne a média. Se a lista estiver vazia, retorne 0.

Desafio — Reescreva o exercício de classificação de prioridade de processos (Etapa 3) usando funções: uma para ler respostas s/n, outra para classificar a prioridade e uma main().

Checklist da Etapa

- ☐ Sei definir e chamar funções.
- ☐ Sei usar parâmetros com valores padrão e argumentos nomeados.
- ☐ Entendo a diferença entre return e print().
- ☐ Sei que variáveis dentro de funções são locais.
- ☐ Escrevo docstrings e nomes de funções com verbos.

Etapa 8 — Strings em profundidade e tratamento de erros

Objetivos da Etapa

- Manipular textos com os principais métodos de string.
- Limpar e padronizar dados digitados pelo usuário.
- Implementar a validação de um CPF.
- Tratar erros com try, except, else e finally.

8.1 Strings são sequências

Um texto se comporta como uma sequência de caracteres. Por isso, tudo o que você aprendeu sobre índices e fatiamento de listas vale para strings:

```
orgao = "Governo do RN"
print(orgao[0])      # G
print(orgao[-2:])    # RN
print(len(orgao))    # 13
print("RN" in orgao) # True
```

◆ VOCÊ SABIA?

Strings são **imutáveis**: os métodos abaixo não alteram o texto original, e sim **devolvem um novo texto**. Por isso você precisa guardar o resultado: **nome = nome.upper()**.

8.2 Métodos essenciais

Método	Exemplo	Resultado
upper() / lower()	"Natal".upper()	"NATAL"
title()	"maria da silva".title()	"Maria Da Silva"
strip()	" RN ".strip()	"RN"
replace(a, b)	"123.456".replace(".", "")	"123456"
split(sep)	"a;b;c".split(";")	["a", "b", "c"]
join(lista)	", ".join(["a", "b"])	"a, b"
startswith(x)	"SEI-2024".startswith("SEI")	True
endswith(x)	"doc.pdf".endswith(".pdf")	True
isdigit()	"12345".isdigit()	True
zfill(n)	"42".zfill(6)	"000042"
find(x)	"Natal/RN".find("/")	5

Método	Exemplo	Resultado
count(x)	"banana".count("a")	3

✓ DICA

Para descobrir todos os métodos de string, digite **dir(str)** no REPL, ou escreva uma variável de texto seguida de ponto no VS Code (nome.) e veja a lista de sugestões aparecer.

8.3 Limpando dados de entrada

★ NO SERVIÇO PÚBLICO

Dados digitados por pessoas chegam com espaços extras, letras maiúsculas e minúsculas misturadas, pontos e traços. Padronizar esses dados antes de salvá-los ou compará-los evita duplicidades e falhas de busca, um problema clássico em cadastros públicos.

```
def limpar_nome(nome: str) -> str:
    """Remove espaços extras e padroniza maiúsculas."""
    return " ".join(nome.split()).title()

def somente_digitos(texto: str) -> str:
    """Mantém apenas os dígitos de um texto (útil para CPF, CEP, telefone)."""
    return "".join(c for c in texto if c.isdigit())

print(limpar_nome("  joSÉ  DA  silva ")) # José Da Silva
print(somente_digitos("529.982.247-25")) # 52998224725
```

✓ DICA

O truque " ".join(texto.split()) é muito útil: split() sem argumentos quebra o texto em qualquer quantidade de espaços, e join() junta tudo com um único espaço. Resultado: espaços duplicados eliminados.

Formatando para exibição

```
def formatar_cpf(cpf: str) -> str:
    d = somente_digitos(cpf)
    return f"{d[:3]}.{d[3:6]}.{d[6:9]}-{d[9:]}"

print(formatar_cpf("52998224725")) # 529.982.247-25
```

✓ DICA

Boa prática em sistemas: **guarde** CPF, CEP e telefone **apenas com dígitos e formate só na hora de exibir**. Isso simplifica buscas e comparações.

8.4 Exemplo completo: validação de CPF

O CPF tem dois **dígitos verificadores**, calculados a partir dos nove primeiros. A validação usa quase tudo o que você aprendeu até aqui: strings, listas, laços, condicionais e funções.

```
def validar_cpf(cpf: str) -> bool:
    """Retorna True se o CPF for válido, com ou sem pontuação."""
    numeros = [int(d) for d in cpf if d.isdigit()]

    # Precisa ter 11 dígitos e não pode ter todos iguais (ex.: 111.111.111-11)
    if len(numeros) != 11 or len(set(numeros)) == 1:
        return False

    # Calcula e confere os dois dígitos verificadores (posições 9 e 10)
    for i in (9, 10):
        soma = sum(numeros[j] * ((i + 1) - j) for j in range(i))
        digito = (soma * 10 % 11) % 10
        if digito != numeros[i]:
            return False
    return True

print(validar_cpf("529.982.247-25")) # True
print(validar_cpf("529.982.247-24")) # False
print(validar_cpf("111.111.111-11")) # False
```

◆ VOCÊ SABIA?

Como funciona o cálculo: para o primeiro dígito, cada um dos 9 primeiros números é multiplicado por pesos de 10 a 2 e os resultados são somados. O dígito é o resto de (soma × 10) dividido por 11; se der 10, vira 0. Para o segundo dígito, repete-se o processo com os 10 primeiros números e pesos de 11 a 2.

⚠ ATENÇÃO

Um CPF **válido** não é necessariamente um CPF **existente** ou regular. A validação apenas confere se os dígitos são matematicamente coerentes, o que já elimina a maioria dos erros de digitação. E lembre-se: CPF é **dado pessoal** protegido pela LGPD. Use CPFs fictícios em exercícios.

8.5 Tratamento de erros (exceções)

Quando algo dá errado durante a execução, o Python lança uma **exceção** e o programa para. Com **try/except**, você pode **capturar** a exceção e decidir o que fazer, em vez de deixar o programa quebrar na frente do usuário.

```
try:
    idade = int(input("Idade: "))
    print(f"Daqui a 10 anos você terá {idade + 10} anos.")
except ValueError:
    print("Por favor, digite um número inteiro.")
```

Se o usuário digitar "trinta", a conversão `int()` lança um **ValueError**, o Python pula para o bloco `except` e mostra a mensagem amigável.

Estrutura completa

```
try:
    dividendo = float(input("Valor total: "))
    divisor = int(input("Dividir entre quantos setores? "))
    resultado = dividendo / divisor
except ValueError:
    print("Valor inválido: digite apenas números.")
except ZeroDivisionError:
    print("Não é possível dividir por zero.")
else:
    print(f"Cada setor recebe R$ {resultado:.2f}") # só roda se não houve erro
finally:
    print("Operação encerrada.") # roda sempre
```

Bloco	Quando é executado
<code>try</code>	Sempre: contém o código que pode falhar
<code>except TipoDeErro</code>	Somente se ocorrer aquele tipo de erro
<code>else</code>	Somente se nenhum erro ocorrer
<code>finally</code>	Sempre, com ou sem erro (útil para liberar recursos)

⚠️ ATENÇÃO

Evite o **except** genérico e silencioso:

```
try:
    processar()
except:
    pass # NÃO FAÇA ISSO: esconde todos os erros, inclusive os seus bugs
```

Capture apenas os erros que você **espera e sabe tratar**. Se precisar capturar qualquer erro, ao menos mostre ou registre a mensagem:

```
try:
    processar()
except Exception as erro:
    print(f'Ocorreu um erro inesperado: {erro}')
```

Combinando com laços: entrada à prova de erros

```
def ler_float(mensagem: str) -> float:
    """Pede um número decimal até o usuário digitar um valor válido.
    Aceita vírgula como separador decimal."""
    while True:
        texto = input(mensagem).replace(",", ".")
        try:
            return float(texto)
        except ValueError:
            print("Valor inválido. Exemplo de formato aceito: 1234,56")
```

✓ DICA

Guarde as funções **ler_float()** e **ler_inteiro()** (da Etapa 7). Você vai reutilizá-las em muitos programas. Na Etapa 10, aprenderá a colocá-las em um módulo próprio para importar sempre que quiser.

Lançando suas próprias exceções

```
def calcular_diarias(dias: int, valor: float) -> float:
    if dias <= 0:
        raise ValueError("A quantidade de dias deve ser positiva.")
    return dias * valor
```

8.6 Os erros mais comuns

A tabela completa com causas e soluções está no **Apêndice C**. Os mais frequentes neste ponto do curso são:

Erro	Causa típica
ValueError	Converter um texto inválido, como <code>int("abc")</code>
TypeError	Operar tipos incompatíveis, como <code>"Idade: " + 30</code>
ZeroDivisionError	Dividir por zero
IndexError	Acessar posição inexistente de lista ou string
KeyError	Acessar chave inexistente de dicionário

Exercícios

Exercício 8.1 — Leia um nome completo e mostre: o nome em maiúsculas, o primeiro nome, o último sobrenome e a quantidade de letras (sem contar espaços).

Exercício 8.2 — Crie a função **validar_cep(cep)** que aceite CEPs com ou sem traço e retorne True se, depois de remover o traço, sobram exatamente 8 dígitos.

Exercício 8.3 — Escreva um programa que peça dois números e mostre a divisão do primeiro pelo segundo, tratando os erros de valor inválido e divisão por zero com mensagens amigáveis. O programa deve repetir a pergunta até conseguir calcular.

Desafio — Crie um programa que leia um número de processo no formato **AAAA/NNNNNN** (ano com 4 dígitos, barra e número com até 6 dígitos) e o padronize, completando o número com zeros à esquerda. Exemplo: "2024/37" vira "2024/000037". Rejeite formatos inválidos.

Checklist da Etapa

- ☐ Conheço os principais métodos de string e sei que eles devolvem um novo texto.
- ☐ Sei limpar e padronizar dados digitados.
- ☐ Entendo a lógica da validação de CPF.
- ☐ Sei usar try, except, else e finally.
- ☐ Não uso except genérico para esconder erros.

Etapa 9 — Manipulação de arquivos (TXT, CSV e JSON)

Objetivos da Etapa

- Ler e escrever arquivos de texto com segurança.
- Trabalhar com caminhos de forma portátil usando pathlib.
- Ler e gerar arquivos CSV, incluindo os padrões do Excel brasileiro.
- Ler e gravar dados em JSON.

9.1 Por que arquivos?

Até agora, todos os dados desapareciam quando o programa terminava. Arquivos permitem **guardar informações** de forma permanente e, sobretudo, **processar dados que já existem**: planilhas exportadas de sistemas, relatórios, bases de dados abertos.

★ NO SERVIÇO PÚBLICO

Este é um dos Etapas de aplicação mais imediata: ler uma exportação em CSV de um sistema, filtrar, somar e gerar um resumo é uma tarefa que muitos servidores fazem manualmente toda semana.

9.2 Lendo e escrevendo arquivos de texto

```
# Escrevendo (o modo "w" cria o arquivo ou APAGA o conteúdo existente)
with open("anotacoes.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Reunião com a equipe de TI\n")
    arquivo.write("Pauta: capacitação em Python\n")

# Lendo o arquivo inteiro
with open("anotacoes.txt", "r", encoding="utf-8") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)

# Lendo linha a linha (melhor para arquivos grandes)
with open("anotacoes.txt", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip()) # strip() remove a quebra de linha do fim
```

Modo	Significado	Se o arquivo existe	Se não existe
"r"	Leitura (padrão)	Lê	Erro FileNotFoundError
"w"	Escrita	APAGA e reescreve	Cria
"a"	Acréscimo (append)	Escreve no final	Cria

✓ DICA

Sempre use **with open(...)**. O with fecha o arquivo automaticamente ao final do bloco, mesmo que ocorra um erro. Arquivos esquecidos abertos podem ficar travados ou perder dados.

⚠ ATENÇÃO

Sempre informe **encoding="utf-8"**. No Windows, sem essa opção, o Python pode usar outra codificação e os acentos aparecem como caracteres estranhos (por exemplo, "SecretÃria" em vez de "Secretária"). É uma das fontes de erro mais comuns ao trabalhar com textos em português.

⚠ ATENÇÃO

O modo **"w"** apaga o conteúdo anterior **sem pedir confirmação**. Para acrescentar informações a um arquivo existente, como um log, use **"a"**.

9.3 Caminhos com pathlib

O módulo **pathlib** trata caminhos de arquivos de forma que funciona igual no Windows e no Linux, e oferece várias operações úteis:

```
from pathlib import Path

pasta = Path("relatorios")
pasta.mkdir(exist_ok=True)          # cria a pasta, se não existir

arquivo = pasta / "janeiro.txt"     # junta caminhos com a barra /
arquivo.write_text("Relatório de janeiro", encoding="utf-8")

print(arquivo.exists())             # True
print(arquivo.name)                 # janeiro.txt
print(arquivo.suffix)               # .txt
print(arquivo.read_text(encoding="utf-8"))

for csv in Path(".").glob("*.csv"): # lista todos os CSVs da pasta atual
    print(csv)
```

✓ DICA

No Windows, caminhos escritos à mão como **"C:\novos\teste.txt"** causam problemas, porque a barra invertida tem significado especial em strings (\n é quebra de linha!). Use **Path** com barras normais, ou prefixe o texto com r: **r"C:\novos\teste.txt"**.

9.4 Arquivos CSV

CSV (*Comma-Separated Values*) é o formato mais comum para trocar dados tabulares entre sistemas. Cada linha é um registro e as colunas são separadas por um delimitador. Suponha o arquivo **despesas.csv**:

```
orgao;categoria;valor
SEAD;Material de escritório;1250.40
SESAP;Medicamentos;58900.00
SEAD;Combustível;3200.00
SEEC;Merenda;41200.75
SESAP;Combustível;7800.00
```

⚠ ATENÇÃO

O Excel em português salva CSV usando **ponto e vírgula (;)** como separador, e não vírgula, porque a vírgula já é o separador decimal no Brasil. Sempre abra o arquivo em um editor de texto para conferir o separador antes de escrever o código, e informe-o com **delimiter=";"**.

Lendo com DictReader

O **csv.DictReader** transforma cada linha em um dicionário, usando o cabeçalho como chaves. É a forma mais legível de trabalhar com CSV:

```
import csv

with open("despesas.csv", encoding="utf-8-sig", newline="") as f:
    leitor = csv.DictReader(f, delimiter=";")
    for linha in leitor:
        print(linha["orgao"], linha["valor"])
```

✓ DICA

Use **encoding="utf-8-sig"** para ler arquivos gerados pelo Excel. Ele remove um caractere invisível (chamado BOM) que o Excel coloca no início do arquivo e que, se não for removido, "gruda" no nome da primeira coluna, causando um **KeyError** misterioso.

⚠ ATENÇÃO

Todos os valores lidos de um CSV chegam como **texto**. Para fazer contas, converta com **float()**. Se os números vierem no formato brasileiro (1.250,40), remova os pontos e troque a vírgula por ponto antes de converter.

Escrevendo com DictWriter

```
import csv

resumo = [
    {"orgao": "SEAD", "total": 4450.40},
    {"orgao": "SESAP", "total": 66700.00},
]

with open("resumo.csv", "w", encoding="utf-8-sig", newline="") as f:
    escritor = csv.DictWriter(f, fieldnames=["orgao", "total"], delimiter=";")
    escritor.writeheader()
    escritor.writerows(resumo)
```

◆ VOCÊ SABIA?

O parâmetro **newline=""** evita que apareçam linhas em branco extras entre os registros no Windows. Use-o sempre ao abrir arquivos CSV.

9.5 Exemplo completo: resumo de despesas por órgão

```
import csv
from pathlib import Path

ARQUIVO = Path("despesas.csv")

def carregar_despesas(caminho: Path) -> list[dict]:
    with open(caminho, encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f, delimiter=";"))

def totalizar_por_orgao(despesas: list[dict]) -> dict:
    totais = {}
    for d in despesas:
        valor = float(d["valor"])
        totais[d["orgao"]] = totais.get(d["orgao"], 0) + valor
    return totais

def main():
    if not ARQUIVO.exists():
        print(f"Arquivo {ARQUIVO} não encontrado.")
        return
    totais = totalizar_por_orgao(carregar_despesas(ARQUIVO))
    ordenados = sorted(totais.items(), key=lambda item: item[1], reverse=True)
    for orgao, total in ordenados:
        print(f"{orgao:<6} R$ {total:>12,.2f}")

main()
```

SESAP	R\$	66,700.00
SEEC	R\$	41,200.75
SEAD	R\$	4,450.40

◆ VOCÊ SABIA?

O trecho **key=lambda item: item[1]** diz ao `sorted()` para ordenar pelo segundo elemento de cada par (o total), e não pelo nome. Uma **lambda** é uma função pequena, sem nome, escrita em uma linha. Não se preocupe em dominá-la agora; basta reconhecer esse padrão de ordenação, que é muito comum.

9.6 Arquivos JSON

JSON é o formato padrão para troca de dados na web e em APIs (você o usará bastante na Etapa 15 e no Django). A boa notícia é que ele se parece muito com os dicionários e listas do Python.

```
import json

config = {
    "orgao": "SEAD",
    "responsavel": "Coordenação de TI",
    "emails_alerta": ["ti@exemplo.rn.gov.br"],
    "limite_diarias": 10,
}

# Gravando
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=4)

# Lendo
with open("config.json", encoding="utf-8") as f:
    dados = json.load(f)
print(dados["orgao"])
```

✓ DICA

ensure_ascii=False mantém os acentos legíveis no arquivo (sem ele, "Coordenação" vira "Coordena\u00e7\u00e3o"). **indent=4** deixa o arquivo organizado e fácil de ler por pessoas.

Python	JSON
dict	objeto { }
list, tuple	array []
str	string
int, float	number
True / False	true / false
None	null

Exercícios

Exercício 9.1 — Crie um programa de "diário de bordo": a cada execução, ele pede uma anotação e a acrescenta, com a data, ao arquivo **diario.txt**, sem apagar as anteriores. Depois, mostra todo o conteúdo do arquivo.

Exercício 9.2 — Usando o arquivo **despesas.csv** deste capítulo, mostre o total gasto com a categoria "Combustível" e a despesa de maior valor (órgão, categoria e valor).

Exercício 9.3 — Crie um programa que salve uma lista de contatos (nome, setor e ramal) em **contatos.json** e outro trecho que leia o arquivo e mostre os contatos de um setor informado pelo usuário.

Desafio — Baixe um conjunto de dados em CSV de um portal de dados abertos (do RN, de um município ou do governo federal). Abra-o em um editor de texto para descobrir o separador e a codificação, e escreva um programa que mostre quantas linhas ele tem, os nomes das colunas e um resumo de alguma coluna numérica.

Checklist da Etapa

- ☐ Uso sempre `with open(...)` e `encoding="utf-8"`.
- ☐ Sei a diferença entre os modos `"r"`, `"w"` e `"a"`.
- ☐ Sei usar `Path` para montar caminhos e verificar se arquivos existem.
- ☐ Sei ler CSV com `DictReader` e `delimiter=";"`.
- ☐ Sei gravar e ler JSON com acentos legíveis.

Etapa 10 — Módulos, pacotes e ambientes virtuais

Objetivos da Etapa

- Importar e usar módulos da biblioteca padrão.
- Trabalhar com datas usando datetime.
- Criar os seus próprios módulos e pacotes.
- Instalar bibliotecas externas com pip.
- Criar e usar ambientes virtuais com venv.
- Organizar a estrutura de um projeto.

10.1 O que são módulos?

Um **módulo** é simplesmente um arquivo .py com funções, variáveis e classes que podem ser **reaproveitadas** em outros programas. O Python vem com centenas de módulos prontos, a chamada **biblioteca padrão**: é por isso que se diz que o Python vem "com pilhas incluídas".

10.2 Formas de importar

```
import math
print(math.sqrt(144))           # 12.0

from math import sqrt, pi      # importa apenas o que vai usar
print(sqrt(81))                # 9.0

import statistics as st        # apelido para nomes longos
print(st.median([3, 1, 4, 1, 5]))
```

⚠ ATENÇÃO

Evite `from modulo import *`. Essa forma importa tudo de uma vez, e fica impossível saber de onde veio cada nome, além de arriscar sobrescrever funções suas sem aviso.

✓ DICA

Coloque todos os import no **início do arquivo**, em três grupos separados por uma linha em branco: primeiro a biblioteca padrão, depois bibliotecas externas e por último os seus próprios módulos.

10.3 Módulos úteis da biblioteca padrão

Módulo	Para que serve	Exemplo
math	Funções matemáticas	<code>math.ceil(4.1) → 5</code>
random	Números e escolhas aleatórias	<code>random.choice(lista)</code>
statistics	Média, mediana, desvio padrão	<code>statistics.mean(notas)</code>
datetime	Datas e horas	<code>date.today()</code>
pathlib	Caminhos e arquivos	<code>Path("dados").exists()</code>
csv / json	Formatos de dados	(Etapa 9)
collections	Estruturas especiais	<code>Counter(lista)</code>
os	Interação com o sistema	<code>os.getenv("USUARIO")</code>
decimal	Contas financeiras exatas	<code>Decimal("0.10")</code>

10.4 Trabalhando com datas

★ NO SERVIÇO PÚBLICO

Datas estão em toda parte no serviço público: prazos processuais, vencimentos de contratos, tempo de serviço, vigência de licenças. O módulo **datetime** resolve os cálculos com segurança, incluindo anos bissextos.

```
from datetime import date, datetime, timedelta

hoje = date.today()
print(hoje)                # 2026-09-30 (formato ISO)
print(hoje.strftime("%d/%m/%Y")) # 30/09/2026 (formato brasileiro)

# Convertendo texto em data
admissao = datetime.strptime("15/03/2010", "%d/%m/%Y").date()

# Diferença entre datas
tempo = hoje - admissao
print(f"Dias de serviço: {tempo.days}")

# Somando dias a uma data (prazo)
prazo = hoje + timedelta(days=15)
print(f"Prazo final: {prazo.strftime('%d/%m/%Y')}")
```

Código	Significado	Exemplo
%d	Dia com 2 dígitos	05
%m	Mês com 2 dígitos	09
%Y	Ano com 4 dígitos	2026
%H:%M	Hora e minuto	14:30

✓ DICA

Memorize a dupla: **strftime** (*string format time*) transforma data **em texto**; **strptime** (*string parse time*) transforma texto **em data**. O "f" é de formatar, o "p" é de *parse* (interpretar).

10.5 Criando seus próprios módulos

Lembra das funções `ler_inteiro()` e `ler_float()`? Coloque-as em um arquivo chamado **utilitarios.py**:

```
# utilitarios.py
"""Funções de apoio para entrada de dados."""

def ler_inteiro(mensagem: str) -> int:
    while True:
        try:
            return int(input(mensagem))
        except ValueError:
            print("Digite um número inteiro.")

def ler_float(mensagem: str) -> float:
    while True:
        try:
            return float(input(mensagem).replace(",", "."))
        except ValueError:
            print("Digite um número válido.")
```

Agora qualquer programa **na mesma pasta** pode usá-las:

```
# programa.py
from utilitarios import ler_inteiro, ler_float

dias = ler_inteiro("Dias: ")
valor = ler_float("Valor da diária: ")
print(f"Total: R$ {dias * valor:.2f}")
```

O famoso `if __name__ == "__main__"`

Quando um arquivo é importado, **todo o código dele é executado**. Se o `utilitarios.py` tiver testes soltos no final, eles vão rodar dentro do programa que o importou. A solução é proteger esse código assim:

```
# utilitarios.py (continuação)
if __name__ == "__main__":
    # Este bloco só roda quando o arquivo é executado diretamente
    # (python utilitarios.py), e NÃO quando é importado.
    print(ler_inteiro("Teste: "))
```

✓ DICA

Adote este padrão em todos os seus programas: coloque a lógica principal em uma função **main()** e termine o arquivo com **if __name__ == "__main__": main()**. Assim, qualquer arquivo seu pode ser importado por outro sem efeitos colaterais.

Pacotes

Um **pacote** é uma pasta que agrupa vários módulos. Basta criar a pasta com um arquivo **__init__.py** (que pode ficar vazio) dentro dela:

```
meu_projeto/
├── main.py
├── ferramentas/
│   ├── __init__.py
│   ├── entrada.py
│   └── validacoes.py
```

```
# main.py
from ferramentas.validacoes import validar_cpf
from ferramentas.entrada import ler_inteiro
```

◆ VOCÊ SABIA?

Essa organização em pacotes é exatamente a que o Django usa. Cada "app" de um projeto Django é um pacote Python, e você reconhecerá essa estrutura no Módulo 3.

10.6 Bibliotecas externas e o pip

Além da biblioteca padrão, existem centenas de milhares de bibliotecas criadas pela comunidade, publicadas no **PyPI** (pypi.org). Elas são instaladas com o **pip**, o gerenciador de pacotes do Python.

```
pip install requests      # instala
pip list                  # lista os pacotes instalados
pip show requests         # detalhes de um pacote
pip uninstall requests    # remove
```

⚠ ATENÇÃO

Instale bibliotecas apenas de fontes confiáveis e confira o nome com cuidado. Existem pacotes maliciosos com nomes parecidos com os de bibliotecas famosas (por exemplo, "requeusts" no lugar de "requests"). Em ambiente corporativo, siga as orientações de segurança da TI do seu órgão.

10.7 Ambientes virtuais (venv)

O problema

Imagine que o **Projeto A** do seu setor usa a versão 4 do Django, e o **Projeto B**, mais novo, usa a versão 5. Se as bibliotecas forem instaladas no Python do sistema, só pode existir **uma versão** de cada. Atualizar para um projeto quebra o outro.

A solução

Um **ambiente virtual** é uma pasta com uma cópia isolada do Python e suas próprias bibliotecas. Cada projeto tem o seu ambiente, como uma caixa de ferramentas exclusiva. O que você instala em um não interfere nos outros nem no sistema.

Passo a passo

Passo 1 — Abra o terminal na pasta do projeto. No VS Code, o terminal já abre na pasta que está aberta no editor.

Passo 2 — Crie o ambiente. Por convenção, a pasta do ambiente se chama **.venv**.

```
python -m venv .venv
```

Passo 3 — Ative o ambiente, conforme o seu sistema:

Sistema / terminal	Comando de ativação
Windows (Prompt de Comando)	.venv\Scripts\activate.bat
Windows (PowerShell)	.venv\Scripts\Activate.ps1
Linux e macOS	source .venv/bin/activate

Quando o ambiente está ativo, o nome **(.venv)** aparece no início da linha do terminal:

```
(.venv) C:\Users\ana\curso-python>
```

Passo 4 — Instale as bibliotecas normalmente com pip. Elas ficarão apenas dentro deste ambiente.

Passo 5 — Para sair do ambiente, digite **deactivate**.

⚠ ATENÇÃO

No **PowerShell** do Windows, a ativação pode falhar com uma mensagem sobre **execução de scripts desabilitada**. Há duas saídas: usar o **Prompt de Comando** (cmd) em vez do PowerShell, ou liberar scripts apenas para o seu usuário com o comando **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned**. Em máquinas corporativas, consulte a TI antes de alterar essa configuração.

✓ DICA

O VS Code detecta o ambiente automaticamente. Se não detectar, tecle **Ctrl+Shift+P**, digite **Python: Select Interpreter** e escolha o Python que está dentro de **.venv**. Com isso, o botão de Play e o terminal passam a usar o ambiente do projeto.

requirements.txt: a receita do ambiente

Para que um colega (ou um servidor de produção) recrie exatamente o mesmo ambiente, registre as bibliotecas e versões em um arquivo **requirements.txt**:

```
pip freeze > requirements.txt      # gera a lista a partir do ambiente atual
```

```
certifi==2025.8.3
charset-normalizer==3.4.3
idna==3.10
requests==2.32.5
urllib3==2.5.0
```

Em outra máquina, basta criar e ativar um novo ambiente e executar:

```
pip install -r requirements.txt
```

Regras de ouro dos ambientes virtuais

- **Um ambiente por projeto.** Crie o **.venv** dentro da pasta do projeto.
- **Sempre ative antes de trabalhar.** Se o **(.venv)** não aparece no terminal, o pip vai instalar no lugar errado.
- **Nunca edite nem copie a pasta .venv.** Ela é descartável: se algo der errado, apague-a e recrie a partir do **requirements.txt**.
- **Não envie a pasta .venv para o controle de versão.** Na Etapa 13 (Git), você aprenderá a ignorá-la com o arquivo **.gitignore**. O que se compartilha é o **requirements.txt**.
- **Atualize o requirements.txt** sempre que instalar ou remover uma biblioteca.

◆ VOCÊ SABIA?

Existem ferramentas mais modernas que automatizam esse fluxo, como o **uv** e o **Poetry**. Elas são ótimas, mas o **venv** com **pip** vem embutido no Python, funciona em qualquer lugar e é o fundamento para entender todas as outras.

10.8 Estrutura recomendada de um projeto

```
consolidador_despesas/
├── .venv/                # ambiente virtual (não versionar)
├── dados/               # arquivos de entrada
│   └── despesas.csv
├── saida/               # arquivos gerados
├── ferramentas/        # seus módulos
│   ├── __init__.py
│   ├── entrada.py
│   └── validacoes.py
├── main.py              # ponto de entrada do programa
├── requirements.txt     # dependências
└── README.md           # explica o que o projeto faz e como executar
```

✓ DICA

Todo projeto deve ter um **README.md** explicando, em poucas linhas, o que ele faz, como instalar e como executar. Pense no colega que vai assumir o projeto quando você estiver de férias.

10.9 Exemplo completo: consultando uma API com requests

Para fechar o módulo, um exemplo que usa uma biblioteca externa dentro de um ambiente virtual. Com o ambiente ativo, instale o requests (**pip install requests**) e execute:

```
import requests

def buscar_municipios_rn() -> list[dict]:
    url = "https://servicodados.ibge.gov.br/api/v1/localidades/estados/RN/municipios"
    resposta = requests.get(url, timeout=10)
    resposta.raise_for_status()      # lança erro se a resposta não for de sucesso
    return resposta.json()

def main():
    try:
        municipios = buscar_municipios_rn()
    except requests.RequestException as erro:
        print(f"Não foi possível consultar a API: {erro}")
        return
    print(f"O RN tem {len(municipios)} municípios.")
    print("Os 5 primeiros em ordem alfabética:")
    for m in sorted(municipios, key=lambda m: m["nome"])[5:]:
        print(f"  - {m['nome']}")

if __name__ == "__main__":
    main()
```

◆ VOCÊ SABIA?

Este exemplo antecipa o Etapa 15, dedicado ao consumo de APIs. Se a rede do seu órgão bloquear o acesso, não se preocupe: o objetivo aqui é praticar o fluxo de ambiente virtual e instalação de pacotes.

Exercícios

Exercício 10.1 — Crie um programa que leia a data de início de um contrato (dd/mm/aaaa) e o prazo de vigência em dias, e mostre a data de término e quantos dias faltam para ela a partir de hoje.

Exercício 10.2 — Crie o pacote **ferramentas** com os módulos **entrada.py** (ler_inteiro, ler_float) e **validacoes.py** (validar_cpf, validar_cep), e um **main.py** que importe e use todas essas funções.

Exercício 10.3 — Crie uma pasta de projeto, um ambiente virtual, ative-o, instale a biblioteca **requests**, gere o requirements.txt e confira o conteúdo do arquivo. Depois, desative o ambiente, apague a pasta .venv e recrie tudo a partir do requirements.txt.

Projeto de encerramento do módulo — Crie o **Consolidador de Despesas**: um projeto organizado conforme a seção 10.8, com ambiente virtual e README.md, que leia todos os arquivos CSV da pasta **dados/**, valide os registros (ignorando e contando as linhas com valores inválidos), totalize as despesas por órgão e por categoria, e grave os resultados em **saida/resumo.csv** e **saida/resumo.json**.

Checklist da Etapa

- ☐ Sei importar módulos das três formas e evito import *.
- ☐ Sei converter datas entre texto e date e calcular prazos.
- ☐ Sei criar meus módulos e uso if __name__ == "__main__".
- ☐ Sei criar, ativar e desativar um ambiente virtual.
- ☐ Sei gerar e usar o requirements.txt.
- ☐ Sei organizar a estrutura de pastas de um projeto.

Apêndice A — Gabarito comentado

As soluções abaixo são **uma** forma de resolver cada exercício, e não a única. Se o seu programa produz o resultado correto e está legível, ele está certo. Compare as abordagens: muitas vezes o gabarito mostra um recurso que você ainda não tinha pensado em usar.

✓ DICA

Tente resolver cada exercício por pelo menos 15 minutos antes de consultar o gabarito. Se travar, olhe só o começo da solução e tente terminar sozinho.

Etapa 1

Exercício 1.1

```
>>> 3200 * 15 / 100
480.0
>>> 30 * 24
720
>>> 2 ** 20
1048576
```

Exercício 1.2

```
print("Nome: Ana Souza")
print("Órgão: Secretaria de Estado da Administração")
print("Setor: Coordenadoria de Recursos Humanos")
```

Exercício 1.3

Primeiro erro: falta fechar as aspas na linha 1 (SyntaxError: unterminated string literal). Segundo erro: **Print** com P maiúsculo não existe (NameError), porque o Python diferencia maiúsculas de minúsculas.

```
print("Secretaria de Administração")
print("Governo do RN")
```

Etapa 2

Exercício 2.1

```
salario = float(input("Salário bruto: R$ "))
percentual = float(input("Reajuste (%): "))
novo_salario = salario * (1 + percentual / 100)
print(f"Novo salário: R$ {novo_salario:.2f}")
```

Exercício 2.2

```
total_minutos = int(input("Minutos: "))
horas = total_minutos // 60
minutos = total_minutos % 60
print(f"{total_minutos} minutos = {horas} hora(s) e {minutos} minuto(s)")
```

Exercício 2.3

```
municipio = input("Município: ")
populacao = int(input("População: "))
print(f"O município de {municipio} tem {populacao} habitantes.")
print(f"10% da população corresponde a {populacao * 0.10:.0f} pessoas.")
```

O formato `:.0f` arredonda para zero casas decimais, já que não faz sentido falar em frações de pessoas.

Etapa 3

Exercício 3.1

```
numero = int(input("Número: "))
if numero % 2 == 0:
    print("Par")
else:
    print("Ímpar")
```

Exercício 3.2

```
horas = float(input("Horas trabalhadas no mês: "))
if horas > 160:
    print(f"Horas extras: {horas - 160}")
else:
    print("Sem horas extras")
```

Exercício 3.3

```
uf = input("Sigla do estado: ").strip().upper()
if uf in ("RN", "PB", "PE", "CE", "AL", "SE", "BA", "PI", "MA"):
    print("Região Nordeste")
else:
    print("Outra região")
```

Converter a entrada com `.upper()` antes de comparar faz o programa aceitar "rn", "Rn" ou "RN" sem precisar testar cada variação.

Etapa 4

Exercício 4.1

```
n = int(input("Tabuada de: "))
for i in range(1, 11):
    print(f"{n} x {i:>2} = {n * i}")
```

Exercício 4.2

```
total = 0
maior = 0
for i in range(1, 6):
    valor = float(input(f"Despesa {i}: R$ "))
    total += valor
    if i == 1 or valor > maior:
        maior = valor
print(f"Total: R$ {total:.2f}")
print(f"Média: R$ {total / 5:.2f}")
print(f"Maior: R$ {maior:.2f}")
```

O teste `i == 1` garante que a primeira despesa sempre se torne a maior inicial, o que funciona mesmo se todos os valores forem negativos ou zero.

Exercício 4.3

```
tentativas = 0
while True:
    senha = input("Senha: ")
    tentativas += 1
    if senha == "gov2026":
        break
    print("Senha incorreta.")
print(f"Acesso liberado após {tentativas} tentativa(s).")
```

⚠ ATENÇÃO

Este exercício é apenas didático. Senhas reais **nunca** devem ser escritas no código-fonte nem comparadas em texto puro. Em sistemas de verdade, como os que você fará com Django, usam-se mecanismos seguros de autenticação.

Etapa 5

Exercício 5.1

```
notas = [7.5, 9.0, 6.0, 8.5, 5.5, 9.5]
media = sum(notas) / len(notas)
acima = [n for n in notas if n > media]
print(f"Maior: {max(notas)} Menor: {min(notas)} Média: {media:.2f}")
print(f"Notas acima da média: {len(acima)}")
```

Exercício 5.2

```
municipios = []
while True:
    nome = input("Município (ou 'fim'): ").strip().title()
    if nome.lower() == "fim":
        break
    if nome not in municipios:
        municipios.append(nome)
for m in sorted(municipios):
    print(m)
```

Exercício 5.3

```
valores = [120, 45, 980, 300, 15, 760]
acima_100 = [v for v in valores if v > 100]
com_acrescimo = [v * 1.10 for v in valores]
print(acima_100)          # [120, 980, 300, 760]
print(com_acrescimo)
```

Etapa 6

Exercício 6.1

```
veiculo = {"placa": "QGA1B23", "modelo": "Pick-up 4x4", "ano": 2022, "km": 45300}
for campo, valor in veiculo.items():
    print(f"{campo}: {valor}")
veiculo["km"] = int(input("Nova quilometragem: "))
print(f"Quilometragem atualizada: {veiculo['km']}")
```

Exercício 6.2

```
viagens = ["Natal", "Mossoró", "Natal", "Caicó", "Natal", "Mossoró"]
contagem = {}
for cidade in viagens:
    contagem[cidade] = contagem.get(cidade, 0) + 1
for cidade, qtd in contagem.items():
    print(f"{cidade}: {qtd} viagem(ns)")
```

Exercício 6.3

```
cadastro = {}
while True:
    matricula = input("Matrícula (Enter para encerrar): ").strip()
    if not matricula:
        break
    cadastro[matricula] = input("Nome: ").strip()

busca = input("Consultar matrícula: ").strip()
print(cadastro.get(busca, "Matrícula não encontrada"))
```

Etapa 7

Exercício 7.1

```
def eh_par(numero: int) -> bool:
    return numero % 2 == 0

print(eh_par(4), eh_par(7))    # True False
```

Não é preciso escrever **if ... return True else return False**: a própria comparação já produz True ou False.

Exercício 7.2

```
def calcular_imc(peso: float, altura: float) -> float:
    return peso / altura ** 2

def classificar_imc(imc: float) -> str:
    if imc < 18.5:
        return "Abaixo do peso"
    if imc < 25:
        return "Normal"
    if imc < 30:
        return "Sobrepeso"
    return "Obesidade"

imc = calcular_imc(70, 1.75)
print(f"IMC {imc:.1f}: {classificar_imc(imc)}")    # IMC 22.9: Normal
```

Exercício 7.3

```
def media_lista(valores: list) -> float:
    if not valores:
        return 0
    return sum(valores) / len(valores)
```

Verificar a lista vazia antes evita um `ZeroDivisionError`, já que `len([])` é 0.

Etapa 8

Exercício 8.1

```
nome = " ".join(input("Nome completo: ").split())
partes = nome.split()
print(nome.upper())
print(f"Primeiro nome: {partes[0]}")
print(f"Último sobrenome: {partes[-1]}")
print(f"Letras: {len(nome.replace(' ', ''))}")
```

Exercício 8.2

```
def validar_cep(cep: str) -> bool:
    cep = cep.strip().replace("-", "")
    return len(cep) == 8 and cep.isdigit()

print(validar_cep("59064-901"))    # True
print(validar_cep("5906490"))      # False
```

Exercício 8.3

```
while True:
    try:
        a = float(input("Dividendo: "))
        b = float(input("Divisor: "))
        print(f"Resultado: {a / b:.2f}")
        break
    except ValueError:
        print("Digite apenas números.")
    except ZeroDivisionError:
        print("O divisor não pode ser zero.")
```

Etapa 9

Exercício 9.1

```
from datetime import datetime

anotacao = input("Anotação: ")
agora = datetime.now().strftime("%d/%m/%Y %H:%M")

with open("diario.txt", "a", encoding="utf-8") as f:
    f.write(f"[{agora}] {anotacao}\n")

with open("diario.txt", encoding="utf-8") as f:
    print(f.read())
```

Exercício 9.2

```
import csv

with open("despesas.csv", encoding="utf-8-sig", newline="") as f:
    despesas = list(csv.DictReader(f, delimiter=";"))

combustivel = sum(float(d["valor"]) for d in despesas
                  if d["categoria"] == "Combustível")
maior = max(despesas, key=lambda d: float(d["valor"]))

print(f"Total com combustível: R$ {combustivel:.2f}")
valor_maior = float(maior["valor"])
print(f"Maior despesa: {maior['orgao']} - {maior['categoria']} - R$ {valor_maior:.2f}")
```

Exercício 9.3

```
import json

contatos = [
    {"nome": "Ana", "setor": "RH", "ramal": "2101"},
    {"nome": "Bruno", "setor": "TI", "ramal": "2250"},
    {"nome": "Clara", "setor": "RH", "ramal": "2102"},
]
with open("contatos.json", "w", encoding="utf-8") as f:
    json.dump(contatos, f, ensure_ascii=False, indent=4)

with open("contatos.json", encoding="utf-8") as f:
    dados = json.load(f)

setor = input("Setor: ").strip().upper()
for c in dados:
    if c["setor"] == setor:
        print(f"{c['nome']} - ramal {c['ramal']}")
```

Etapa 10

Exercício 10.1

```
from datetime import date, datetime, timedelta

inicio = datetime.strptime(input("Início (dd/mm/aaaa): "), "%d/%m/%Y").date()
prazo = int(input("Vigência (dias): "))

termino = inicio + timedelta(days=prazo)
restantes = (termino - date.today()).days

print(f"Término: {termino.strftime('%d/%m/%Y')}")
if restantes >= 0:
    print(f"Faltam {restantes} dia(s).")
else:
    print(f"Contrato vencido há {-restantes} dia(s).")
```

Exercício 10.2

Estrutura de pastas:

```
exercicio_10_2/
├── main.py
├── ferramentas/
│   ├── __init__.py
│   ├── entrada.py      # ler_inteiro e ler_float (seção 10.5)
│   └── validacoes.py   # validar_cpf (Etapa 8) e validar_cep (Exercício 8.2)
```

```
# main.py
from ferramentas.entrada import ler_inteiro, ler_float
from ferramentas.validacoes import validar_cpf, validar_cep

def main():
    idade = ler_inteiro("Idade: ")
    salario = ler_float("Salário: ")
    cpf = input("CPF: ")
    cep = input("CEP: ")
    print(f"Idade {idade}, salário R$ {salario:.2f}")
    print("CPF válido" if validar_cpf(cpf) else "CPF inválido")
    print("CEP válido" if validar_cep(cep) else "CEP inválido")

if __name__ == "__main__":
    main()
```

Exercício 10.3

```
mkdir exercicio_10_3
cd exercicio_10_3
python -m venv .venv
.venv\Scripts\activate.bat      (Linux/macOS: source .venv/bin/activate)
pip install requests
pip freeze > requirements.txt
deactivate
rmdir /s /q .venv               (Linux/macOS: rm -rf .venv)
python -m venv .venv
.venv\Scripts\activate.bat
pip install -r requirements.txt
```

Ao final, **pip list** deve mostrar exatamente os mesmos pacotes de antes. Esse é o teste que comprova que o `requirements.txt` é uma "receita" confiável do ambiente.

Apêndice B — Atalhos essenciais do VS Code

Os atalhos abaixo são para Windows e Linux. No macOS, troque **Ctrl** por **Cmd**.

Atalho	Ação
Ctrl+S	Salvar o arquivo
Ctrl+'	Abrir ou fechar o terminal integrado
Ctrl+Shift+P	Paleta de comandos (acesso a qualquer função do editor)
Ctrl+P	Abrir rapidamente um arquivo pelo nome
Ctrl+/ Tab / Shift+Tab	Comentar ou descomentar as linhas selecionadas
Alt+↑ / Alt+↓	Aumentar ou diminuir a indentação da seleção
Shift+Alt+↓	Mover a linha atual para cima ou para baixo
Shift+Alt+↓	Duplicar a linha atual
Ctrl+D	Selecionar a próxima ocorrência da palavra (edição múltipla)
Ctrl+F / Ctrl+H	Localizar / Localizar e substituir
F2	Renomear uma variável ou função em todo o arquivo
F12	Ir para a definição de uma função
F5	Iniciar a depuração
Ctrl+Espaço	Mostrar sugestões de preenchimento
Shift+Alt+F	Formatar o documento

✓ DICA

Se aprender apenas três atalhos, fique com **Ctrl+/
Tab / Shift+Tab** (comentar), **Ctrl+Shift+P** (paleta de comandos) e **F2** (renomear). Eles economizam muito tempo desde o primeiro dia.

Apêndice C — Guia de mensagens de erro

Lembre-se: leia o Traceback **de baixo para cima**. A última linha diz o **tipo** do erro; as anteriores dizem **onde** ele ocorreu.

Erro	O que significa	Causas mais comuns	Como resolver
SyntaxError	O código não segue as regras da linguagem	Aspas ou parênteses sem fechar; falta de dois-pontos no if, for, def	Confira a linha indicada e a linha anterior a ela
IndentationError	Recuo incorreto	Bloco sem recuo; mistura de Tab e espaços	Use sempre 4 espaços; peça ao VS Code para converter a indentação
NameError	Nome não definido	Erro de digitação; variável usada antes de ser criada; maiúscula/minúscula	Confira a grafia e a ordem das linhas
TypeError	Operação com tipo inadequado	Somar texto com número; chamar função com argumentos errados	Converta com int(), float() ou str(); use f-strings
ValueError	Tipo certo, valor inválido	int("abc"); float("1.234,50")	Valide ou limpe a entrada; use try/except
IndexError	Posição inexistente	Índice maior que o tamanho da lista; lista vazia	Confira len(); lembre que o índice começa em 0
KeyError	Chave inexistente no dicionário	Erro de digitação na chave; BOM no CSV	Use get(); abra CSV do Excel com utf-8-sig
AttributeError	O objeto não tem esse método	Método de outro tipo; variável que virou None	Confira o tipo com type(); veja se a função tem return
ZeroDivisionError	Divisão por zero	Média de lista vazia; divisor digitado como 0	Verifique o divisor antes de dividir
FileNotFoundError	Arquivo não encontrado	Nome errado; programa executado em outra pasta	Use Path(...).exists(); confira a pasta do terminal
ModuleNotFoundError	Módulo não encontrado	Biblioteca não instalada; ambiente virtual não ativado	Ative o .env e instale com pip
UnicodeDecodeError	Problema de codificação	Arquivo em codificação diferente de UTF-8	Informe encoding="utf-8" ou tente "latin-1"

✓ DICA

Ao pedir ajuda a um colega, ao monitor ou em fóruns, envie sempre: (1) o trecho de código, (2) a **mensagem de erro completa**, (3) o que você esperava que acontecesse e (4) o que você já tentou. Uma boa pergunta costuma ser respondida muito mais rápido.

Apêndice D — Glossário

Termo	Definição
Algoritmo	Sequência finita de passos para resolver um problema.
API	Interface que permite a um programa conversar com outro, geralmente pela internet, trocando dados em JSON.
Argumento	Valor enviado a uma função no momento da chamada.
Biblioteca	Conjunto de módulos prontos que oferecem funcionalidades reutilizáveis.
Booleano	Tipo de dado que só assume True (verdadeiro) ou False (falso).
Bug	Defeito no programa que causa comportamento incorreto.
CSV	Formato de arquivo de texto para dados em tabela, com colunas separadas por um delimitador.
Depuração (debug)	Processo de encontrar e corrigir bugs.
Dicionário	Estrutura que guarda pares de chave e valor.
Docstring	Texto de documentação escrito logo no início de uma função, classe ou módulo.
Exceção	Erro que ocorre durante a execução e pode ser tratado com try/except.
f-string	Texto prefixado com f que permite inserir valores entre chaves.
Função	Bloco de código nomeado e reutilizável.
Indentação	Reco no início da linha, que em Python define os blocos de código.
Iteração	Cada repetição de um laço.
JSON	Formato de texto para troca de dados, parecido com dicionários e listas do Python.
Lista	Coleção ordenada e mutável de valores.
Módulo	Arquivo .py que pode ser importado por outros programas.
Pacote	Pasta que agrupa módulos e contém um arquivo <code>__init__.py</code> .
Parâmetro	Variável declarada na definição de uma função para receber valores.
pip	Gerenciador de pacotes do Python.
PyPI	Repositório oficial de bibliotecas Python (pypi.org).
REPL	Modo interativo do Python, identificado pelo sinal <code>>>></code> .
Script	Programa em um arquivo, geralmente usado para automatizar tarefas.
String	Tipo de dado que representa texto.
Traceback	Relatório de erro que mostra onde e por que o programa falhou.
Tupla	Coleção ordenada e imutável de valores.
Variável	Nome que aponta para um valor na memória.
venv	Ambiente virtual: instalação isolada do Python com bibliotecas próprias para cada projeto.

Apêndice E — Para continuar estudando

Materiais gratuitos recomendados

- **Documentação oficial do Python** (docs.python.org/pt-br/): inclui um tutorial completo em português.
- **Curso em Vídeo — Python** (Gustavo Guanabara): videoaulas didáticas em português, ótimas para reforçar os fundamentos.
- **Python Brasil** (python.org.br/): comunidade brasileira, com links para grupos, eventos e materiais.
- **Real Python** (realpython.com/): artigos aprofundados em inglês sobre praticamente todos os temas.
- **Exercism** e **Beecrowd**: plataformas com centenas de exercícios com correção automática.

Rotina de estudo sugerida

Quando	O que fazer	Tempo
Antes da Etapa	Ler rapidamente o capítulo da semana	15 min
Depois da Etapa	Refazer os exemplos digitando, sem copiar	30 min
Ao longo da semana	Resolver os três exercícios de fixação	20 min/dia
Fim de semana (opcional)	Tentar o desafio e revisar o checklist	30 a 60 min

O que vem a seguir

No **Módulo 2 — Python Intermediário e Ferramentas**, você aprenderá orientação a objetos, controle de versão com Git, automação de planilhas, consumo de APIs e bancos de dados com SQL. Todos esses temas se apoiam diretamente no que foi visto aqui, e prepararão o terreno para o **Módulo 3**, em que você construirá um sistema web completo com Django.

★ NO SERVIÇO PÚBLICO

Comece a olhar para o seu trabalho com "olhos de programador": qual tarefa repetitiva você faz toda semana? Consolidar planilhas, conferir listas, renomear arquivos, calcular prazos? Anote-a. Ela pode virar o seu projeto final do curso.